

TECHNICAL UNIVERSITY OF MOLDOVA
FACULTY OF COMPUTERS, INFORMATICS, AND MICROELECTRONICS
DEPARTMENT OF SOFTWARE ENGINEERING AND AUTOMATION
SOFTWARE ENGINEERING STUDY PROGRAMME

PROJECT REPORT

BlockSign - Securing Documents With Blockchain

Team No. 1

Team members:

Student 1:	Alexei Pavlovschii, FAF-231
Student 3:	Gabriel Moraru, FAF-232
Student 4:	Filip Obrijan, FAF-232
Student 5:	Vladimir Vitcovschii, FAF-231

TUM

Internship Coordinator:	Elena Gogoi
--------------------------------	-------------------

Submission date: 23/01/2026

Chişinău, 2026

ABSTRACT

The project titled **”BlockSign - Securing Documents With Blockchain”** was developed by students Alexei Pavlovschii, Gabriel Moraru, Filip Obrijan, and Vladimir Vitcovschii from the Technical University of Moldova.

This project comprises 5 chapters: Problem Framing, Domain Analysis, Solution Proposal, System Design and Practical Implementation, as well as Introduction, Conclusions and Bibliography.

It addresses the growing need for secure, transparent, and verifiable document management systems in modern organizations. Traditional approaches often rely on centralized storage and password-based authentication, which are vulnerable to tampering, unauthorized access, and human error. Our solution proposes a **blockchain-anchored mechanism** for document signing and verification, ensuring both the integrity and authenticity of documents.

The system implements a **passwordless authentication** scheme based on cryptographic key pairs, reducing risks associated with compromised credentials. Document integrity is preserved by calculating a SHA-256 hash, while signatures from all involved participants are verified using the **Ed25519 algorithm**. Once finalized, a document’s payload can be anchored to a blockchain network, providing immutable proof of its existence and content at a given time.

From an architectural perspective, the back-end is developed in Node.js with Express.js and Prisma ORM on top of a PostgreSQL database. Email-based workflows, including OTP verification and document notifications, ensure usability and trust between system participants. The modular design of the application allowed the integration with Amazon S3 cloud storage and expanded blockchain support.

Through this project, the students combined concepts from distributed systems, **cryptography**, and software engineering to design and implement a practical, real-world solution. The outcome is a **secure and scalable platform** that demonstrates how blockchain technology can enhance document authentication and verification processes in both academic and enterprise contexts.

Keywords: Blockchain, Cryptography, Documents, Security.

Content

INTRODUCTION	6
1 PROBLEM FRAMING	7
1.1 Problem Description	7
1.2 Problem Statement	10
2 Domain Analysis	11
2.1 Target Audience	11
2.1.1 Individual Users	11
2.1.2 Educational Institutions	11
2.1.3 Small and Medium Enterprises (SMEs)	12
2.1.4 Government Institutions	12
2.1.5 Law Firms and Notary Services	12
2.1.6 Financial and Banking Sectors	13
2.1.7 Healthcare Organizations	13
2.2 Market Size and Growth	13
2.2.1 Key Competitors and Solutions	14
2.2.2 Trends and Opportunities	15
2.2.3 Challenges and Barriers	16
2.3 Technical Research	16
2.3.1 Blockchain-Based Notarization	17
2.3.2 Electronic Signatures with Multi-Factor Authentication	18
2.3.3 Electronic & Remote Notarization	18
2.3.4 Advanced Cryptographic Techniques	19
2.3.5 Hybrid and Scalable Architectures	20
3 Solution Proposal	21
4 System Design	23
4.1 Technical Requirements	23
4.1.1 Functional Requirements	23
4.1.2 Non-Functional Requirements	25
4.2 Behavioral Modeling	26
4.2.1 Use Case Diagrams	26
4.2.2 Sequence Diagrams	29
4.3 Structural Modeling	32

4.3.1	Class Diagram	32
4.4	Figma User Interface Mockups	33
5	Practical Implementation	40
5.1	Back-end architecture	40
5.1.1	Application Layer	40
5.1.2	Database Layer	40
5.1.3	Security Measures	41
5.1.4	Authentication and Tokens	41
5.1.5	Email Service	42
5.1.6	Cryptographic Module	42
5.1.7	Document Workflow	43
5.1.8	Extensibility	43
5.2	Front-end UI	43
5.2.1	Frontend Architecture	44
5.2.2	Landing Page and User Onboarding	44
5.2.3	User role specified UI	46
5.2.4	Additional registration steps	47
5.2.5	Document Management Interface	50
5.2.6	Responsive Design and User Experience	51
5.2.7	Client-Side Security Implementation	52
5.2.8	Internationalization and Localization	52
5.2.9	Feature Accessibility by User Role	52
5.2.10	Technology Integration and Performance	53
5.3	iOS Mobile Application	53
5.3.1	Mobile Architecture	54
5.3.2	Authentication and Security	54
5.3.3	Document Management	56
5.3.4	Document Verification	59
5.3.5	User Interface Design	60
5.3.6	Settings and Account Management	60
5.3.7	Offline Capabilities and Caching	61
5.3.8	Error Handling and User Feedback	61
5.3.9	App Store Compliance	62
5.4	DevOps & Deployment	62

5.4.1	Cloud Infrastructure	62
5.4.2	Mobile Application Deployment	63
5.4.3	Containerization Strategy	63
5.4.4	Continuous Integration and Deployment	64
5.4.5	Database Management	64
5.4.6	Environment Configuration	65
5.4.7	Monitoring and Logging	65
5.4.8	Security Hardening	66
5.4.9	Scalability Considerations	66
5.4.10	Disaster Recovery	67
CONCLUSIONS		68
BIBLIOGRAPHY		70

INTRODUCTION

In the digital era, organizations and individuals are increasingly reliant on electronic documents for communication, collaboration, and decision-making. The authenticity, integrity, and accessibility of these documents have therefore become critical factors in building trust and ensuring compliance with regulatory standards. Traditional paper-based approaches to document management are costly, time-consuming, and vulnerable to fraud or human error. At the same time, centralized digital systems often face challenges related to security, data breaches, and limited transparency.

This project addresses these challenges by designing and implementing a secure, scalable, and user-friendly system for document registration, signing, and verification. The system relies on public-key cryptography (Ed25519) to provide a passwordless authentication mechanism that ensures only authorized users can access or sign documents. A carefully designed workflow guides users through registration and approval, while administrators maintain oversight and control of pending registration requests.

The central contribution of this work is the integration of digital signatures and cryptographic payloads into the document lifecycle. Each document is uniquely identified by its SHA-256 hash, which guarantees integrity and allows participants to independently verify that the file they review is identical to the file stored in the system. By requiring each participant to sign the canonical payload, the system ensures non-repudiation and establishes a verifiable chain of consent. Once all signatures are collected, the document status transitions to signed, and participants are notified accordingly.

In addition to the cryptographic foundation, the system emphasizes practical usability. Users interact with the platform through intuitive registration and login flows, email-based verification, and role-based access controls. Administrators can approve or reject registration requests, while users can create documents, tag participants by username, and track their signature status. The platform is designed to integrate with decentralized storage such as Amazon S3 and blockchain anchoring like Polygon, extending its guarantees of persistence and transparency.

Overall, this project demonstrates how modern cryptographic primitives and structured workflows can be combined to deliver a robust document signing and verification solution. It lays the foundation for further research into decentralized trust, compliance with international standards, and seamless integration with existing enterprise systems.

1 PROBLEM FRAMING

1.1 Problem Description

Securing documents—whether physical or digital—remains a persistent, multi-dimensional problem that affects governments, enterprises, and individuals.

Physical Documents: Physical records (e.g., passports, certificates, contracts) have long been protected via signatures, seals, stamps, and locked storage, yet they remain vulnerable to forgery, misuse, and destruction (Figure 1.1). Law-enforcement bodies warn that advances in consumer printing and imaging have lowered the barrier to document counterfeiting [1]. Border and customs activity shows the problem is active and measurable: in one U.S. port alone, officials intercepted more than 6,800 fraudulent or stolen documents in FY2023—a 219% year-over-year increase—demonstrating both scale and growth [2]. In addition to fraud, disasters can irreparably damage archives; professional guidance exists precisely because fires, floods, and collapses have destroyed unique holdings (e.g., the 2009 Cologne City Archive collapse that obliterated a major European archive) [3], [4].



Figure 1.1 - Signature Forgery Impact on Important Processes

Digital Documents: Digitization improved accessibility and scale but introduced remote theft, silent manipulation, mass leakage, and extortion (Figure 1.2). The scale is well documented: Verizon’s 2024 DBIR analyzed 30,458 incidents with 10,626 confirmed breaches, noting increased vulnerability exploitation via web applications [5]. Financial impact remains high: IBM’s 2025 study reports a global average breach cost of USD 4.4M (down from USD 4.88M in 2024, still historically elevated), underscoring that document exposure is expensive even when containment improves [6], [7]. Ransomware and data-theft extortion continue to pressure organizations across critical sectors, with complaints to the FBI’s

IC3 rising 9% in 2024 and losses hitting \$16.6B across cyber and scam crimes [8].

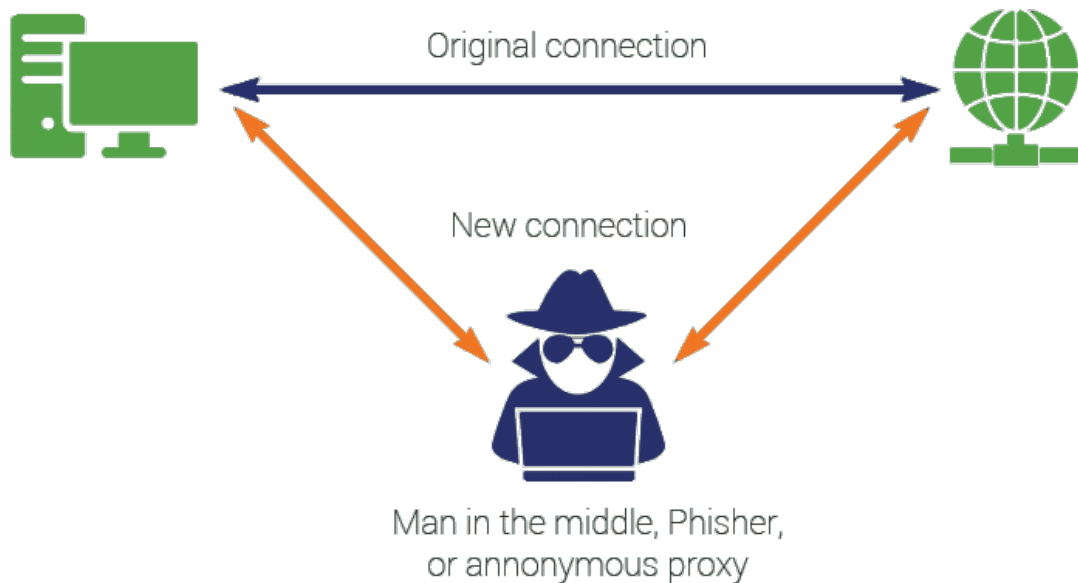


Figure 1.2 - Man In The Middle Attack

Trust and authenticity weaknesses in current ecosystems: Even when digital signatures and PKI are used, centralized trust anchors can fail. The DigiNotar breach (2011) led to hundreds of fraudulent certificates for prominent domains, breaking the authenticity guarantees expected from TLS/PKI and forcing browser vendors to distrust the CA entirely—an instructive “single point of failure” for trust [9]. The episode shows that document authenticity that depends on a compromised authority can be globally undermined.

Human and process error remain a major source of breaches: Beyond external attackers, misconfigurations, errors, and insider misuse contribute materially to breaches. Verizon’s 2024 DBIR details the role of internal actors and errors across sectors; in healthcare, for example, internal actors feature far more prominently than elsewhere, reversing earlier trends [10]. This reflects a broad, persistent problem: authorized access used improperly can compromise sensitive documents at scale.

Regulatory pressure - confidentiality, integrity, accountability: Regulatory frameworks require clear safeguards for documents that include personal data. The GDPR mandates “appropriate technical and organizational measures” and emphasizes data integrity and accountability [11]. In the EU, eIDAS sets legal scaffolding for electronic identification and trust services, defining requirements for trustworthy digital interactions (e.g., signatures, seals, timestamps) [12]. Non-compliance introduces legal and financial risks on top of the technical ones.

Common vulnerability patterns in applications that handle documents: Applications that store, view, transfer, or sign documents routinely exhibit high-impact weaknesses. The OWASP Top 10 highlights recurring problems such as Broken Access Control (ranked #1 in 2021), Cryptographic Failures, Injection, and Security Misconfiguration (Figure 1.3); notably, 94% of tested apps exhibited some form of access control weakness in the dataset behind the 2021 list [13], [14]. These patterns map directly to risks for document confidentiality, integrity, and availability.

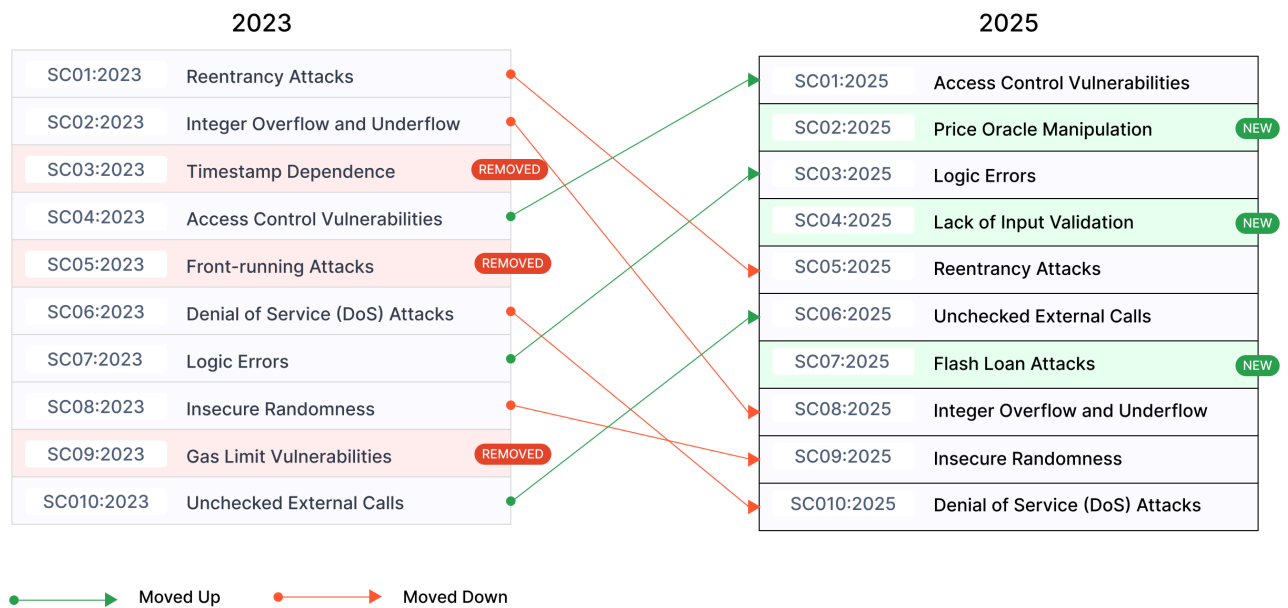


Figure 1.3 - OWASP Top 10 (2023 vs 2025)

The Core Problem: Taken together, these issues demonstrate that document security is a multi-faceted challenge:

- Physical documents face forgery and catastrophic loss risks despite traditional controls [1], [2], [3], [4].
- Digital documents face breach, extortion, and manipulation at global scale, with material financial impact [5], [6], [7], [8].
- Trust infrastructures (e.g., CAs) can become single points of failure [9].
- Human/organizational errors and app-level vulnerabilities remain prevalent [10], [13], [14].
- Regulatory frameworks demand provable safeguards and accountability [11], [12].

These facts collectively demonstrate that ensuring authenticity, integrity, confidentiality, availability, and accountability for documents is an unresolved, real-world problem spanning both physical and digital realms.

1.2 Problem Statement

Existing mechanisms for securing physical and digital documents remain vulnerable to forgery, loss, unauthorized access, manipulation, and systemic trust failures, while regulatory obligations require stronger assurance and accountability. The problem is to guarantee authenticity, integrity, confidentiality, availability, and verifiability of documents over time and across domains, despite evolving threats, human error, and infrastructural weaknesses (Figure 1.4).

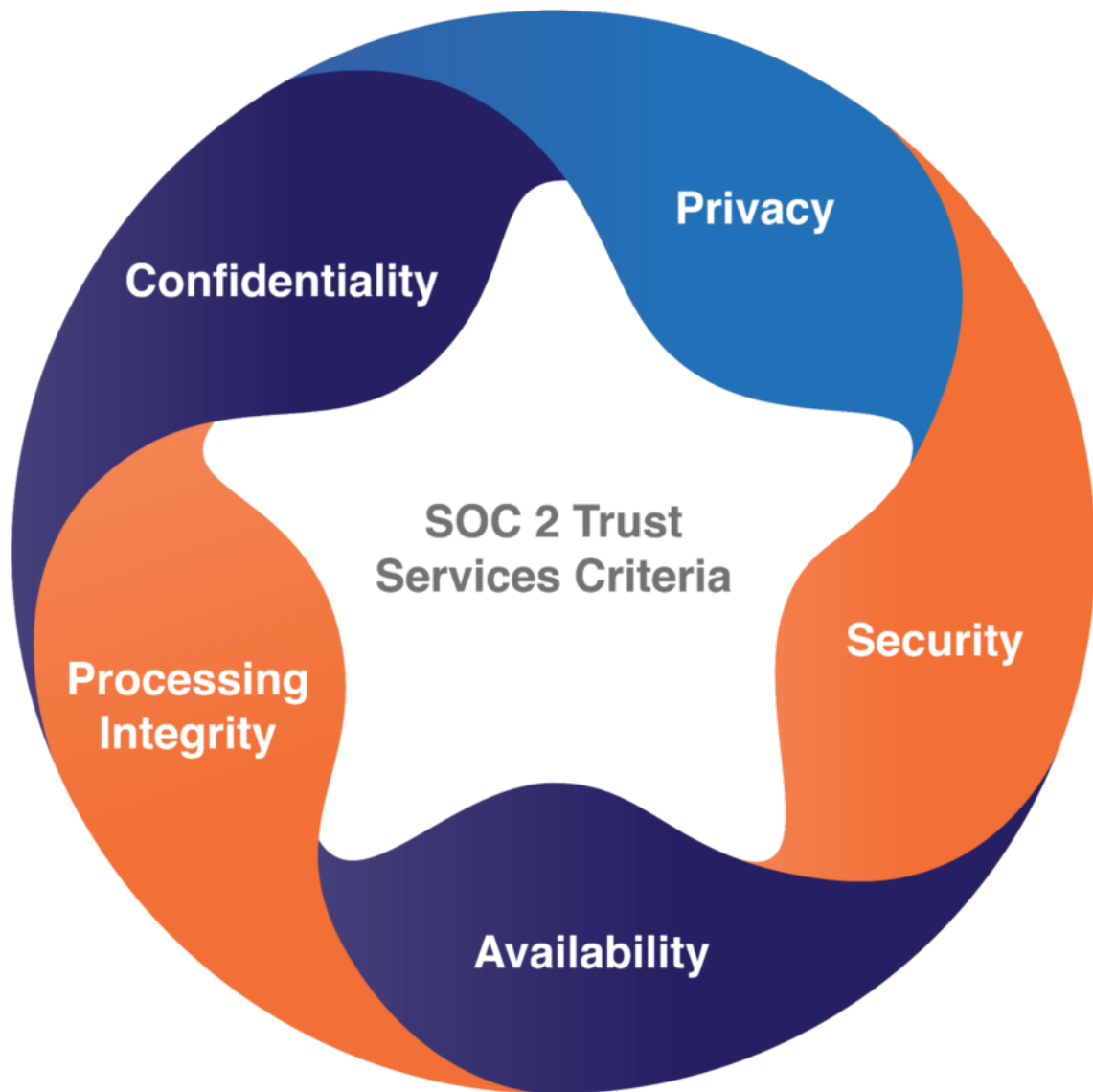


Figure 1.4 - Service Organization Controls (SOC) security principles standard

2 Domain Analysis

2.1 Target Audience

BlockSign is designed to serve a diverse range of users who require secure, verifiable, and legally recognized methods of document handling. By leveraging blockchain technology, BlockSign ensures authenticity, immutability, and compliance across various sectors. The primary audiences are described in this section.

2.1.1 Individual Users

In an era where personal data misuse and identity theft are prevalent, individuals can utilize BlockSign to notarize personal documents such as wills, rental agreements, and contracts. This ensures their documents remain immutable and verifiable over time, providing peace of mind and legal assurance [15].

2.1.2 Educational Institutions

Educational institutions require secure systems for managing sensitive records, including diplomas, transcripts, and certificates (Figure 2.1). Cases of diploma fraud have increased globally, undermining student mobility and employer trust. Blockchain notarization of educational credentials ensures their authenticity across borders, which is particularly important in student exchange programs and international hiring [16].



Figure 2.1 - Digital Certificates

2.1.3 Small and Medium Enterprises (SMEs)

SMEs often lack access to enterprise-grade solutions like DocuSign or Adobe Sign due to cost constraints. BlockSign offers a cost-effective and transparent notarization platform, enabling startups, freelancers, and growing firms to establish trust with partners, clients, and regulators without heavy infrastructure investments [17].

2.1.4 Government Institutions

Government agencies responsible for issuing and verifying official records (e.g., passports, permits, licenses) face persistent threats from forgery and manipulation of physical and digital documents (Figure 2.2). These institutions require trustworthy digital notarization solutions to comply with regulations such as the EU's eIDAS Regulation and the General Data Protection Regulation (GDPR) [18].



Figure 2.2 - Customs and Border Protection officer verifying a travel document

2.1.5 Law Firms and Notary Services

Law firms and notary services handle contracts, affidavits, and property transactions, all of which require high trust and legal enforceability. Traditional notarization can be time-consuming and limited by geographic restrictions. Remote Online Notarization (RON), already gaining traction in the United States and parts of Europe, demonstrates that secure video-based validation combined with blockchain immutability can streamline legal workflows while retaining compliance [19].

2.1.6 Financial and Banking Sectors

Financial institutions, including banks and insurance companies, must guarantee the validity of signed contracts, loan agreements, and client identities (Figure 2.3). Financial institutions are among the industries with the highest breach costs, averaging USD 4.44 million per incident [20].



Figure 2.3 - Personal Loan Agreement

2.1.7 Healthcare Organizations

The healthcare sector is increasingly dependent on electronic records and digital consent forms. The sector faces both internal and external threats: 70% of healthcare breaches involve insiders misusing access privileges [21]. For hospitals, clinics, and research institutions, BlockSign offers a mechanism to notarize patient records, safeguard sensitive data, and ensure compliance with confidentiality obligations under GDPR.

2.2 Market Size and Growth

According to industry reports, the global market for digital signatures was worth about USD 7.47 billion in 2023, and it's expected to soar to around USD 37.79 billion by 2029, expanding at an annual growth rate of 31%.

Other estimates are even more bullish: one study puts the 2023 market at USD 4.6 billion with a forecast to USD 43.5 billion by 2030, representing a 37.9% CAGR between 2023 and 2030 (Figure 2.4) [22]. Another source projects growth from USD 3.2 billion in 2021 to USD 48.4 billion by 2028, at

35.4% CAGR (Figure 2.5) [23]. Despite slight variations in numbers, the consensus is clear: the market is booming.

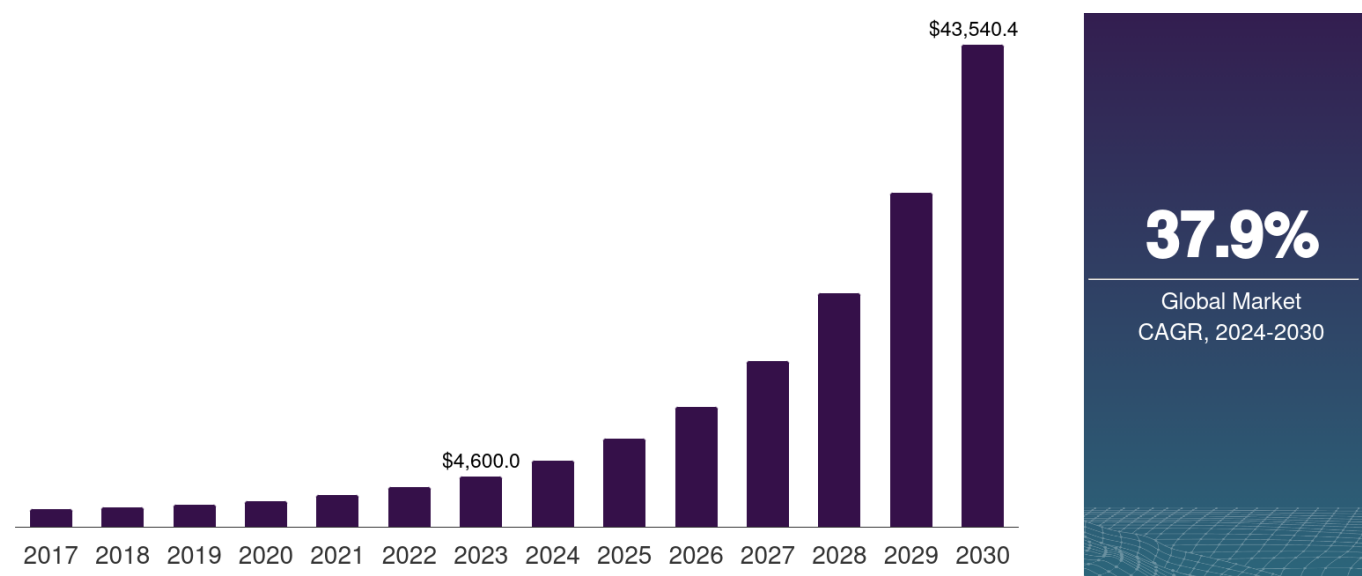


Figure 2.4 - Global Digital Signature Market Size 2017-2030 (Horizon)

Blockchain-related identity services and notarization tools are also on the rise, particularly in industries where trust, compliance, and speed are critical: banking, finance, legal, and healthcare. These solutions help reduce fraud, accelerate cross-border processes, and ensure compliance with regulations like GDPR, eIDAS in Europe, or E-SIGN and UETA in the United States.

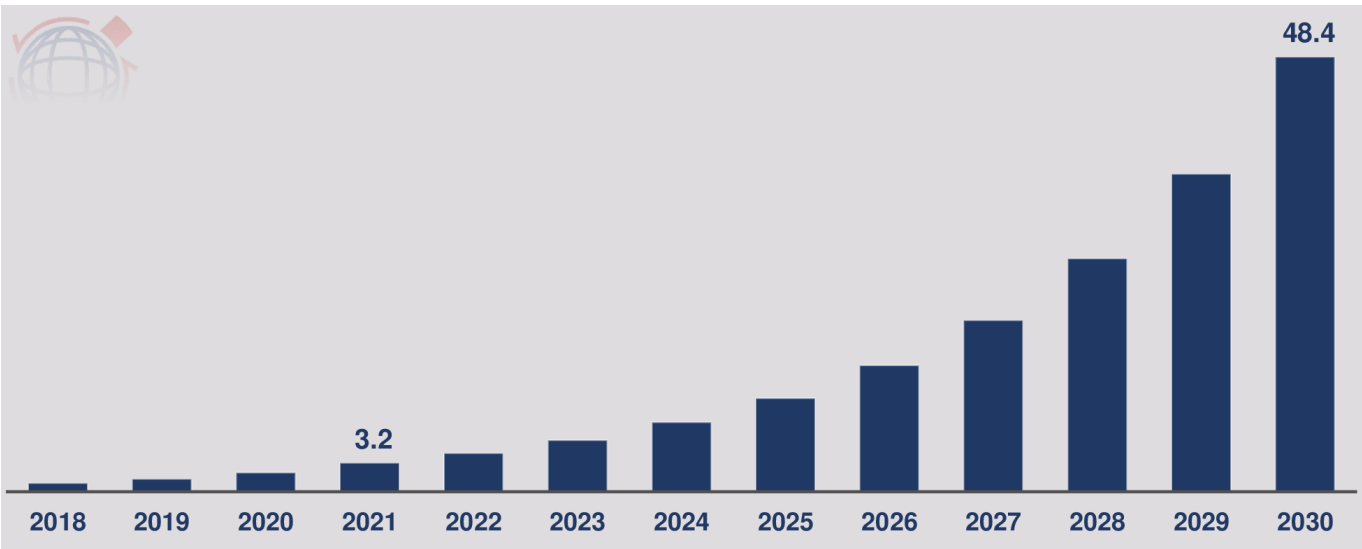


Figure 2.5 - Digital Signature Market 2018-2030 (Acumen)

2.2.1 Key Competitors and Solutions

Several big players are driving the digital signature space:

- **DocuSign** and **Adobe Sign** are trusted and widely used, especially by enterprises, but still rely on

centralized infrastructure.

- **Notarize**, **NotaryCam**, and **SignNow** specialize in Remote Online Notarization (RON), often using video identity checks to meet legal and compliance needs.
- On the blockchain side, platforms like **DoxyChain** and **Blocknotary** offer immutable timestamping and decentralized storage.
- Emerging frameworks like **KILT Protocol** and **Concordium** support decentralized identity and can integrate into notarization workflows.

These players show how mature the e-signature market is becoming—and how blockchain is carving out its place as a serious value-add technology (Figure 2.6) [24].

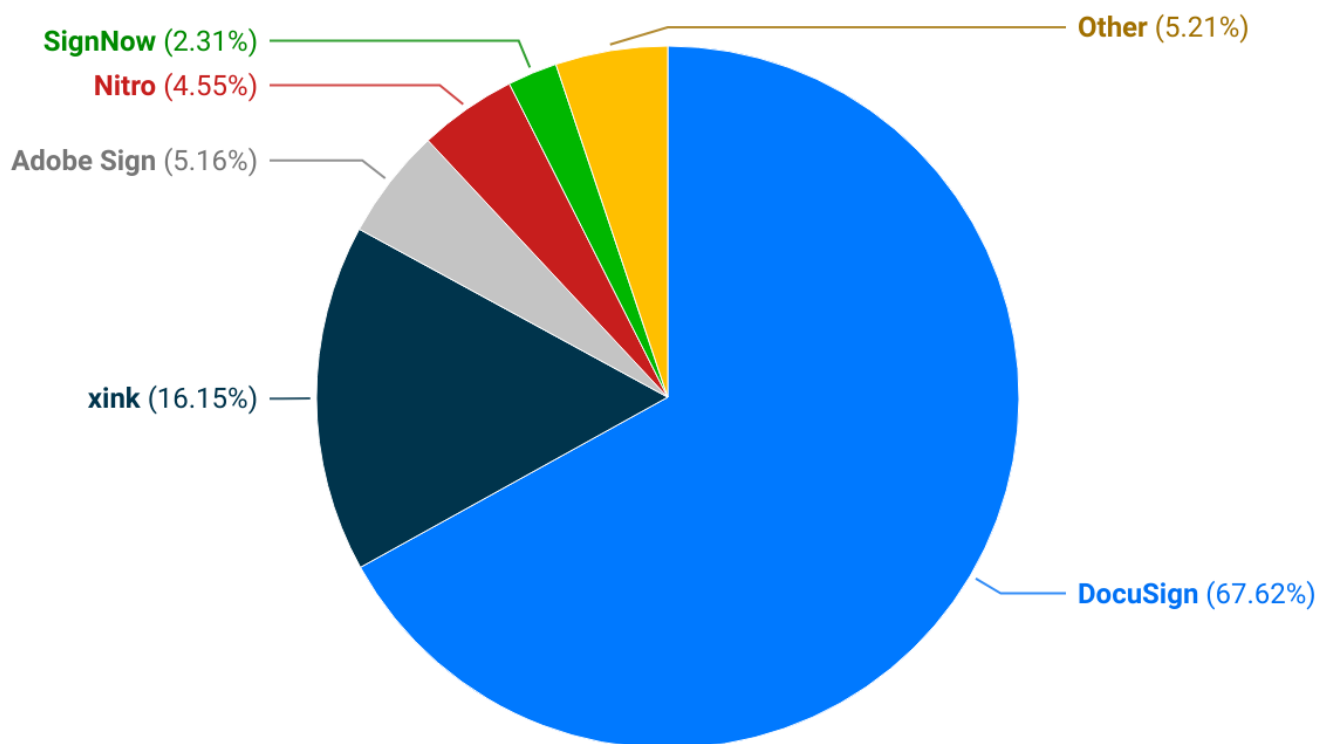


Figure 2.6 - Market Share of Leading Electronic Signature Companies

2.2.2 Trends and Opportunities

A few clear trends are shaping the future of digital signatures:

1. **Trustless, decentralized verification** – Blockchain-based notarization helps remove single points of failure tied to central Certificate Authorities (CAs).
2. **Remote and cross-border notarization** – With more people working globally, demand increases for legally recognized remote notarization systems.
3. **AI and automation integration** – Automated document checks, fraud detection, and smart contract workflows are on the rise.
4. **Quantum-safe cryptography** – Some platforms are starting to prepare for future cryptographic threats

to ensure long-term document integrity.

5. **Hybrid architectures** – Combining public and permissioned blockchains helps balance trust, scalability, and cost-efficiency.

These trends create fertile ground for innovation, especially for solutions that balance security, legal compliance, and user-friendliness (Figure 2.7).

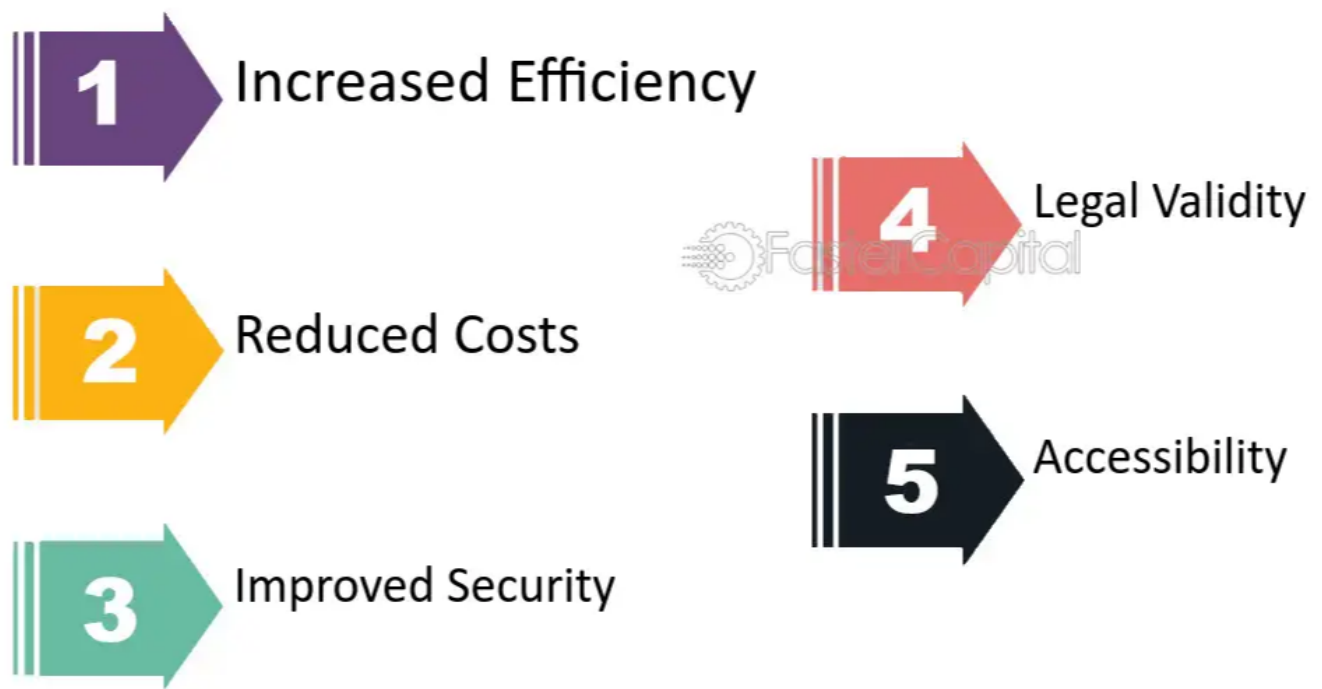


Figure 2.7 - Benefits of Digital Signatures

2.2.3 Challenges and Barriers

However, adoption still faces some challenges:

- Legal frameworks vary widely across countries, slowing global standardization and interoperability.
- Many institutions remain comfortable with traditional notarization; change can feel risky.
- Balancing high security with simplicity is tough — systems that are too complex won't encourage adoption.
- Some blockchains can struggle with scalability and can be costly for frequent use.

2.3 Technical Research

This section describes core technologies potentially applicable for implementing the secure document notarization idea. For each, it will be described how it operates, its benefits, and real-world usage.

2.3.1 Blockchain-Based Notarization

Document notarization on blockchain typically involves computing a cryptographic hash of the file and storing it timestamped on a decentralized ledger. Anyone can later rehash the document to verify integrity without revealing the document's content [25]. Some valuable benefits of this approach are:

- **Immutability and transparency:** Hash entries cannot be altered after being written to the chain and are visible across the network (Figure 2.8).
- **No central point of failure:** Decentralized consensus removes reliance on a single trusted authority [26].
- **Timeproof:** Timestamps on transactions provide verifiable evidence of document existence at a given time [25].

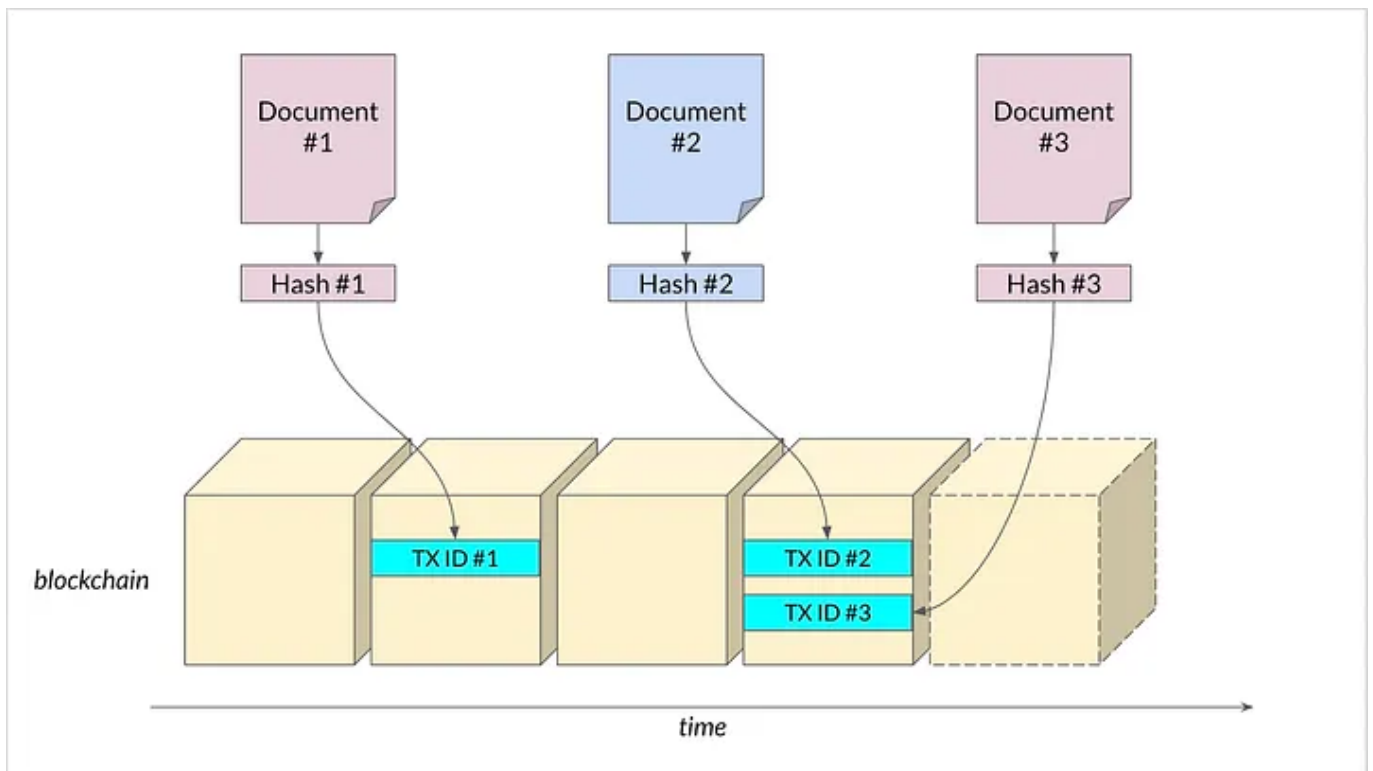


Figure 2.8 - Blockchain Notarization

Recent academic research explores moving from simple “proof of existence” to full document validation, integrating smart contracts, Zero-Knowledge Proofs (ZKPs), AI-based content analysis, and quantum-resistant cryptography—to ensure privacy, detect fraud, and future-proof security [27]. Hybrid blockchain architectures are also being proposed to improve scalability and cost-efficiency, balancing public and permissioned ledger benefits [28].

2.3.2 Electronic Signatures with Multi-Factor Authentication

Combining digital signatures with MFA requires users to authenticate via multiple channels—e.g., password plus a one-time code or biometric—before signing or notarizing a document (Figure 2.9) [29]. Several useful features and benefits are:

- **Significantly enhanced security:** Microsoft reports that MFA can block 99.2% of account compromise attacks [30].
- **Fraud reduction:** Additional authentication steps make forging or stealing identities substantially harder [29], [31].
- **Regulatory compliance:** MFA aids in meeting stringent identity verification requirements in sectors like finance and legal services [29].



Figure 2.9 - Multi-Factor Authentication

Remote Online Notarization (RON) platforms integrate MFA to verify signer identity and prevent unauthorized access—combining this with tamper-evident seals and audit trails strengthens legal validity and trust [32].

2.3.3 Electronic & Remote Notarization

E-Notary systems use digital signatures, cryptographic seals, and a public key infrastructure (PKI) to notarize documents electronically. Remote Online Notarization (RON) adds a layer of video verification and identity validation via secure video + digital credentialing (Figure 2.9) [33]. The benefits include:

- **Tamper-evident documents:** Digital signatures and seals prevent unnoticed alterations.
- **Cost savings and efficiency:** Eliminates paper handling and on-site meetings; notarizations can be completed online, rapidly.

- **Audit trails:** Many systems log each step of the notarization process, boosting transparency and legal compliance.



Figure 2.10 - Remote Notarization

For example, Notario.org offers an online notarization platform supporting identity verification via video, digital certificates, and legal compliance across several countries (Spain, USA, Germany, etc.) [34].

2.3.4 Advanced Cryptographic Techniques

Zero-Knowledge Proofs (ZKPs) enable one party to prove a statement is true without revealing the underlying data. In blockchain notarization, ZKPs can allow verification of document integrity or content compliance without exposing sensitive data (Figure 2.9) [35]. The two benefits of this technique are:

- **Privacy preservation:** Keeps personal or confidential document content hidden while still proving authenticity.
- **Regulatory advantages:** Complies with data protection regulations by limiting exposure of sensitive data.

Zero Knowledge Proofs



Figure 2.11 - Zero-Knowledge Proof Mechanism

With the potential arrival of quantum computing, future document notarization systems must adopt quantum-safe cryptographic algorithms to remain secure long-term. Research highlights this as an evolving domain in notarization systems to guard against future computational threats.

2.3.5 Hybrid and Scalable Architectures

To address limitations of public blockchains (e.g., transaction fees, latency) and private blockchains (e.g., trust, centralization), hybrid DLT (Distributed Ledger Technology) designs are emerging. They combine local/private storage for efficiency with public chains for decentralization and transparency. Benefits include:

- **Cost optimization:** Keeps frequent notarizations low-cost by batching hashing operations off-chain while anchoring checkpoints on-chain.
- **Scalability:** Manages large volumes efficiently while maintaining tamper evidence and verifiability.

3 Solution Proposal

Based on defined problems and domain research, it is evident that existing document management approaches face serious limitations. Physical records remain vulnerable to forgery, loss, and destruction, while digital records suffer from risks such as unauthorized modification, cyberattacks, and dependence on centralized authorities. Current protections, including digital signatures and certificate-based systems, mitigate some threats but are not sufficient to guarantee authenticity, long-term verifiability, and compliance with modern security standards. At the same time, regulatory frameworks such as GDPR and eIDAS demand solutions that ensure confidentiality, integrity, and accountability. These findings highlight the need for a system that not only secures documents but also provides transparency, privacy, and user trust.

The proposed solution is a blockchain-based notarization platform that enables the creation, signing, and verification of documents in a secure and transparent manner. The platform leverages blockchain to store the payload of documents, ensuring immutability and tamper resistance without exposing sensitive content. This approach allows users to instantly verify document authenticity by matching a locally computed hash with the records on the blockchain, making the verification process fast, reliable, and independent of intermediaries.

The architecture of the system is structured around four main components:

1. **Client Applications (Web and Mobile)** – provide users with an accessible interface to create, sign, and verify documents. Both platforms support multilingual functionality (English, Russian, Romanian) and integrate usability features such as search, filtering, and history tracking.
2. **Authentication and Identity Management** – user authentication is handled through challenge-response registration flows, with strong security policies and multi-factor authentication (FaceID) applied during sensitive operations such as signing. Identity verification relies on a one-time check of personal identifiers (e.g., IDNP), which are not stored long term, ensuring compliance with privacy regulations. Session management is secured with JWT tokens.
3. **Application Server** – coordinates document workflows, including creation, pending approvals, and signature collection. The server computes document hashes, manages participant approvals, and interacts with both the blockchain network and the database. It also enforces security controls such as input validation, secure APIs, and proper session handling.
4. **Blockchain and Database Layer** – the blockchain serves as a public, immutable ledger for document hashes, timestamps, and participant references, ensuring integrity and transparency. The database stores only operational metadata and user account information, excluding sensitive identity data. Together, these layers guarantee both system efficiency and data protection.

Functionally, the platform supports a full document lifecycle: registration and identity verification, creation and upload of documents, participant assignment, approval workflows, signing, and final notarization on the blockchain (Figure 3.1). Each step is backed by security controls, ensuring that fraudulent activity, tampering, or unauthorized access is either prevented or immediately detectable.

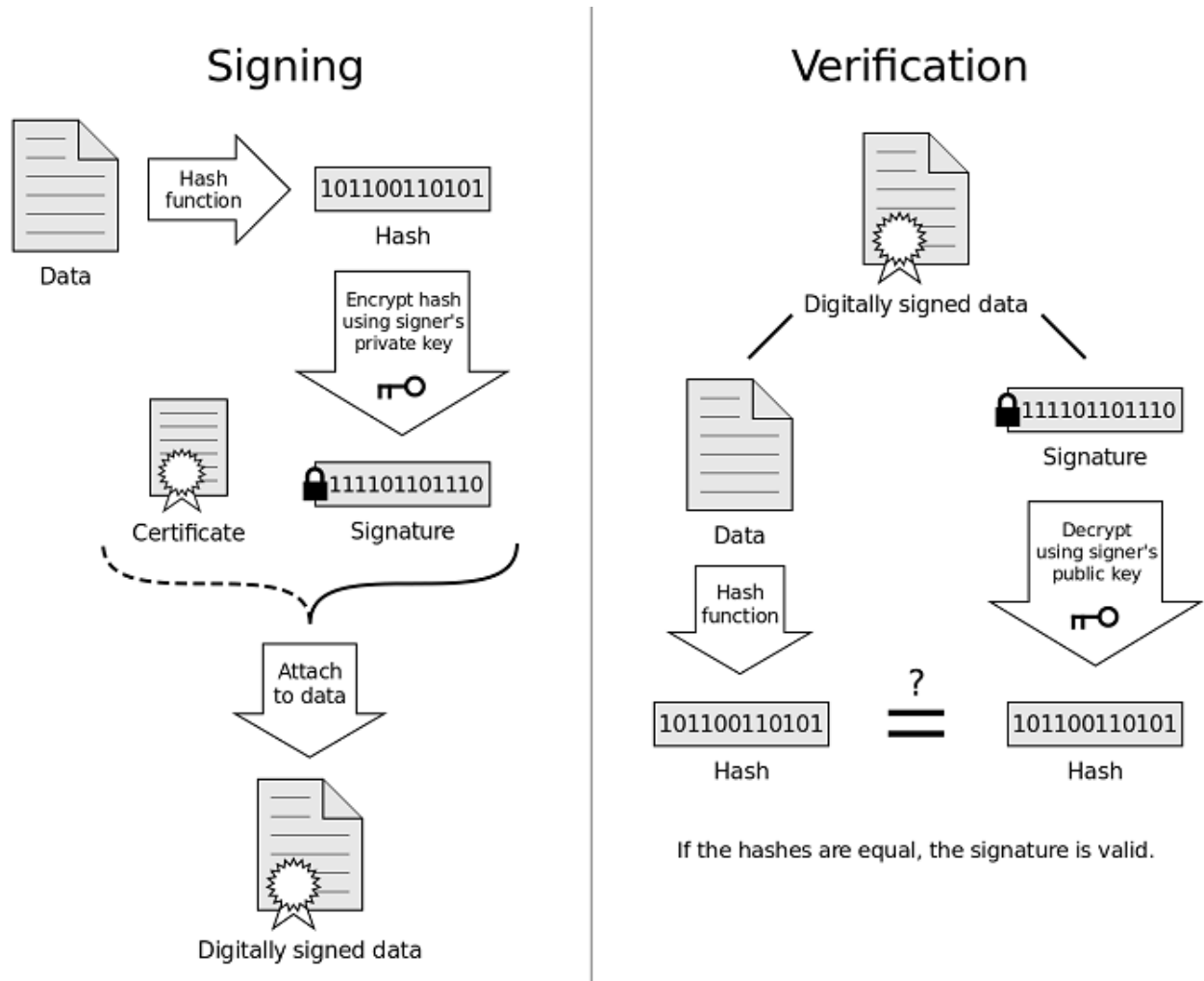


Figure 3.1 - Core Signing and Verifying Logic

By combining usability, regulatory compliance, and cryptographic trust mechanisms, the solution directly addresses the shortcomings identified in traditional document management. It provides a system that ensures authenticity, integrity, transparency, and privacy, offering users a verifiable and tamper-proof method of managing critical documents.

4 System Design

4.1 Technical Requirements

This section describes the architectural backbone of the project, mainly Functional and Non Functional requirements. It describes key actors, functionalities accessed by them, as well as other technical specifications which will be considered in the project practical implementation part (Figure 4.1).

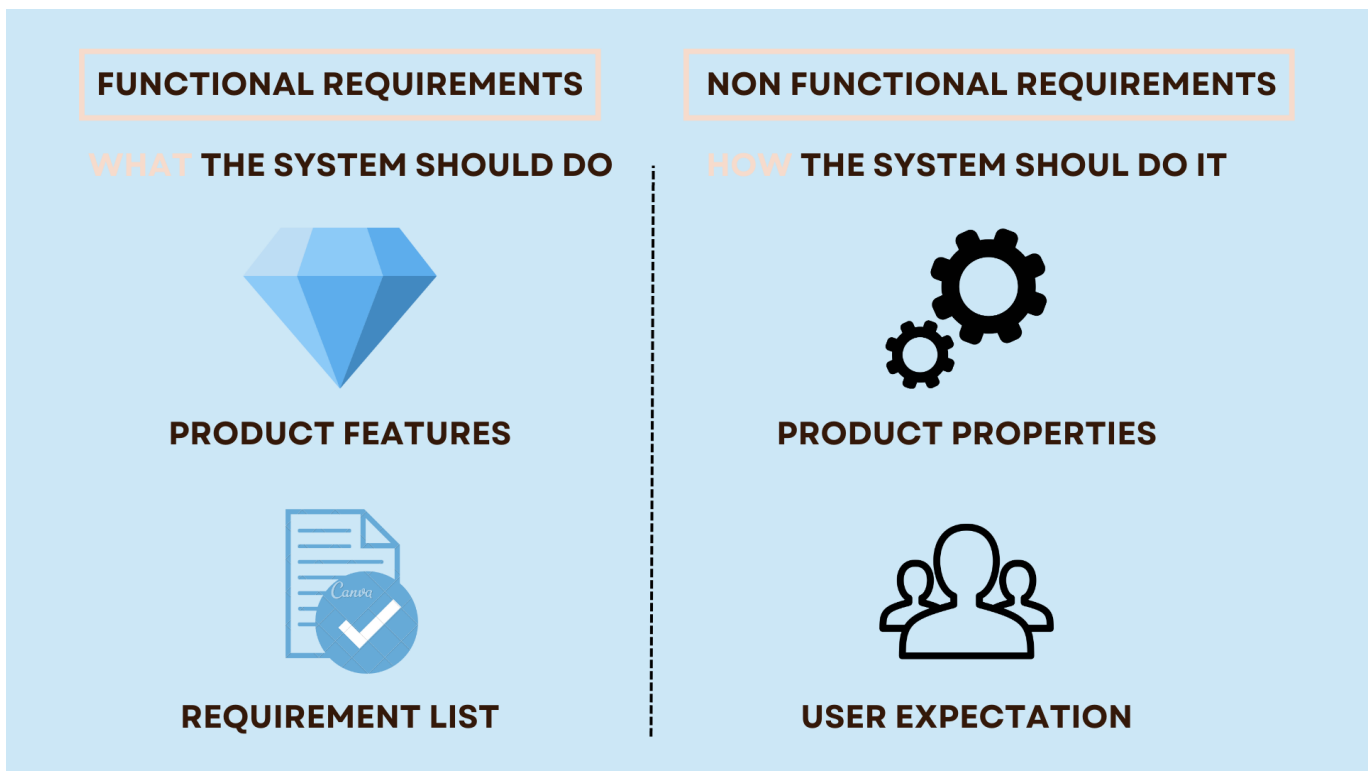


Figure 4.1 - Functional And Non-Functional Requirements

4.1.1 Functional Requirements

The application is designed around three primary user roles:

1. **Unauthorized users**, who may access the main page, perform registration or login, as well as verify a document.
2. **Authorized users**, who gain full access to document-related services such as creation, signing, and account management.
3. **Administrators**, who are responsible for managing the user database and can access limited user information when necessary.

The system incorporates a wide set of functions to support these roles. First, it provides multilingual support, ensuring that the entire interface is available in English, Russian, and Romanian. The main page allows navigation to registration, login, and account pages, as well as a mail integration for sending quick

requests to support.

On the documents page, authorized users may create new documents by uploading files through a drag-and-drop interface, specifying the parties involved, and submitting them for signing. Pending documents appear in the personal cabinet of each participant until all signatures are collected. At the same time, users can verify documents by uploading a file. System generates the hash, and performs a search to check for related blockchain records. If no match is found, a notification is displayed.

The account page contains a full history of signed and pending documents. Each entry provides a name, status, date, owner, participants, signature, and the option to download & view. Account management includes editing personal details and enabling multi-factor authentication.

Additional functionality is provided through search and filters, which allow users to locate documents based on keywords, status, or metadata. Pending documents can be opened in a PDF viewer for review and signed electronically. Once the final party signs, the system prepares the payload of the document and records it immutably on the blockchain.

Finally, the system enforces secure authorization and registration flows. Users may authenticate on any device through secure challenge-response mechanism (Figure 4.2).

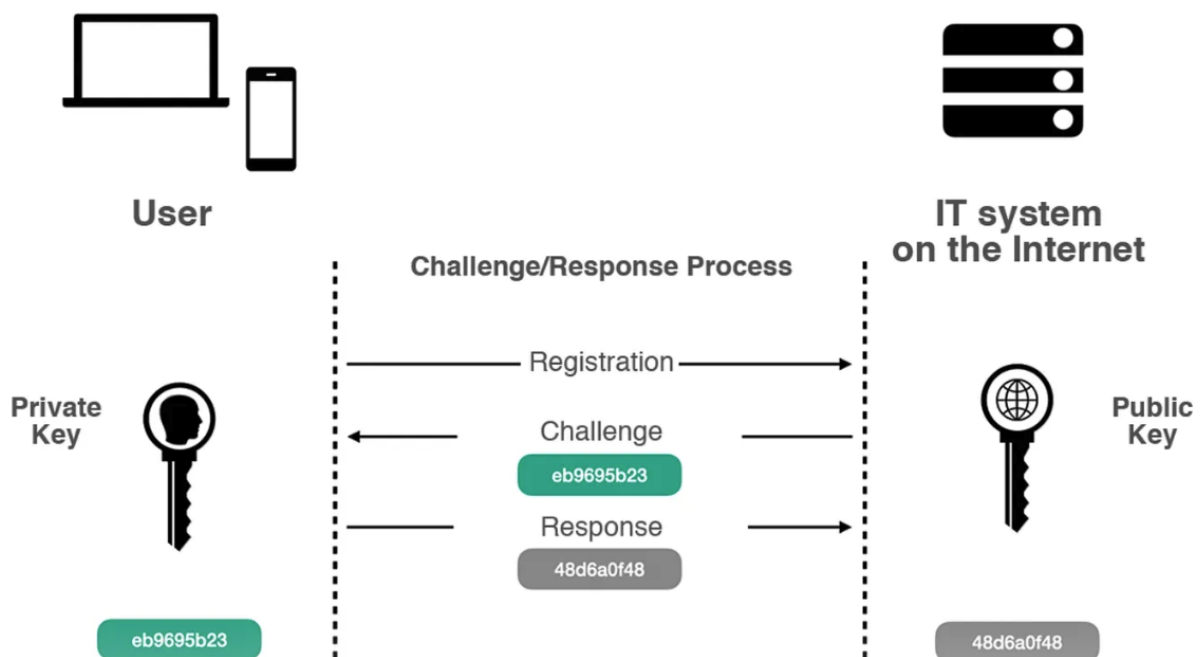


Figure 4.2 - Challenge-Response Mechanism

Registration follows a structured, multi-step process: (1) email verification through a six-digit code, (2) user data entry, and (3) submission of identity data (IDNP, birth date, selfie) for administrative approval.

4.1.2 Non-Functional Requirements

Beyond functionality, the application is governed by several non-functional requirements that ensure its performance, usability, and security.

From a performance perspective, documents themselves are never stored on the blockchain. Instead, only their payload is recorded. This guarantees integrity verification without exposing sensitive content or overloading the ledger.

Regarding usability, the system is designed as both a web application and a mobile application, ensuring broad accessibility. To support a diverse user base, the entire interface is available in three languages: English, Russian, and Romanian.

In terms of security, the platform adopts multiple protective measures:

1. **Sensitive data minimization** – personal identifiers such as IDNP are used solely during registration for verification and are not retained long-term.
2. **Multi-factor authentication (MFA)** – required for signing documents, adding an extra layer of identity assurance.
3. **JWT tokens** – used to secure session management and provide controlled access to the personal cabinet (Figure 4.3).

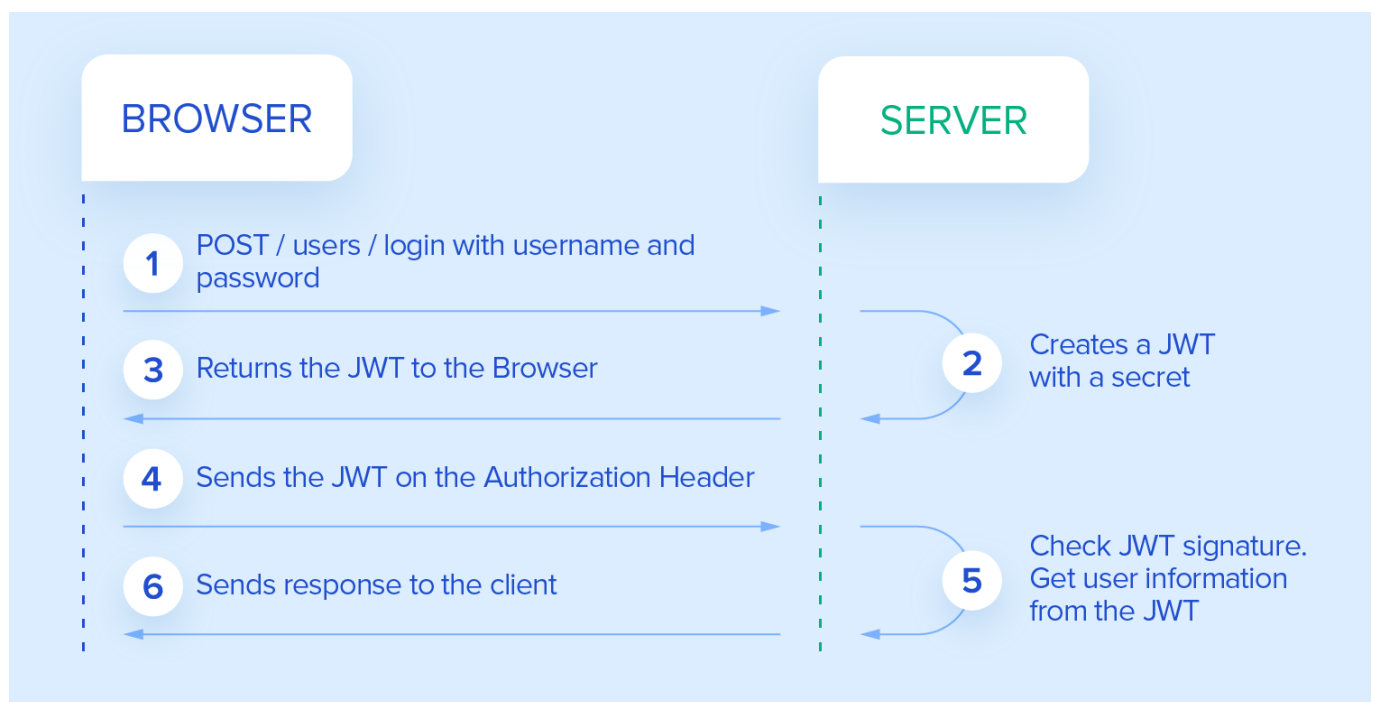


Figure 4.3 - JWT Token Usage Mechanism

Finally, the system adheres to privacy and compliance standards. The blockchain stores only document payload, with no personal data or file content, ensuring confidentiality and compliance with data protection principles.

4.2 Behavioral Modeling

To capture the dynamic aspects of the system and illustrate how users and components interact, several UML diagrams were created. These diagrams help visualize workflows, user interactions, and message flows between system elements, making the architecture more comprehensible and easier to validate against requirements.

4.2.1 Use Case Diagrams

The use case diagrams represent the primary interactions between different user roles and the system, highlighting the functionalities available to each category of user. There are four main use cases described:

1. **Registration process:** The Figure 4.4 illustrates the registration process for an unauthenticated user. The actor initiates the flow by selecting the option to register into the system. The process includes several mandatory steps such as filling in registration data, providing personal information, verifying the email, and awaiting administrator approval. The diagram highlights the dependencies between activities using «include» relationships, showing that registration is a structured, multi-step procedure designed to ensure security and identity verification.

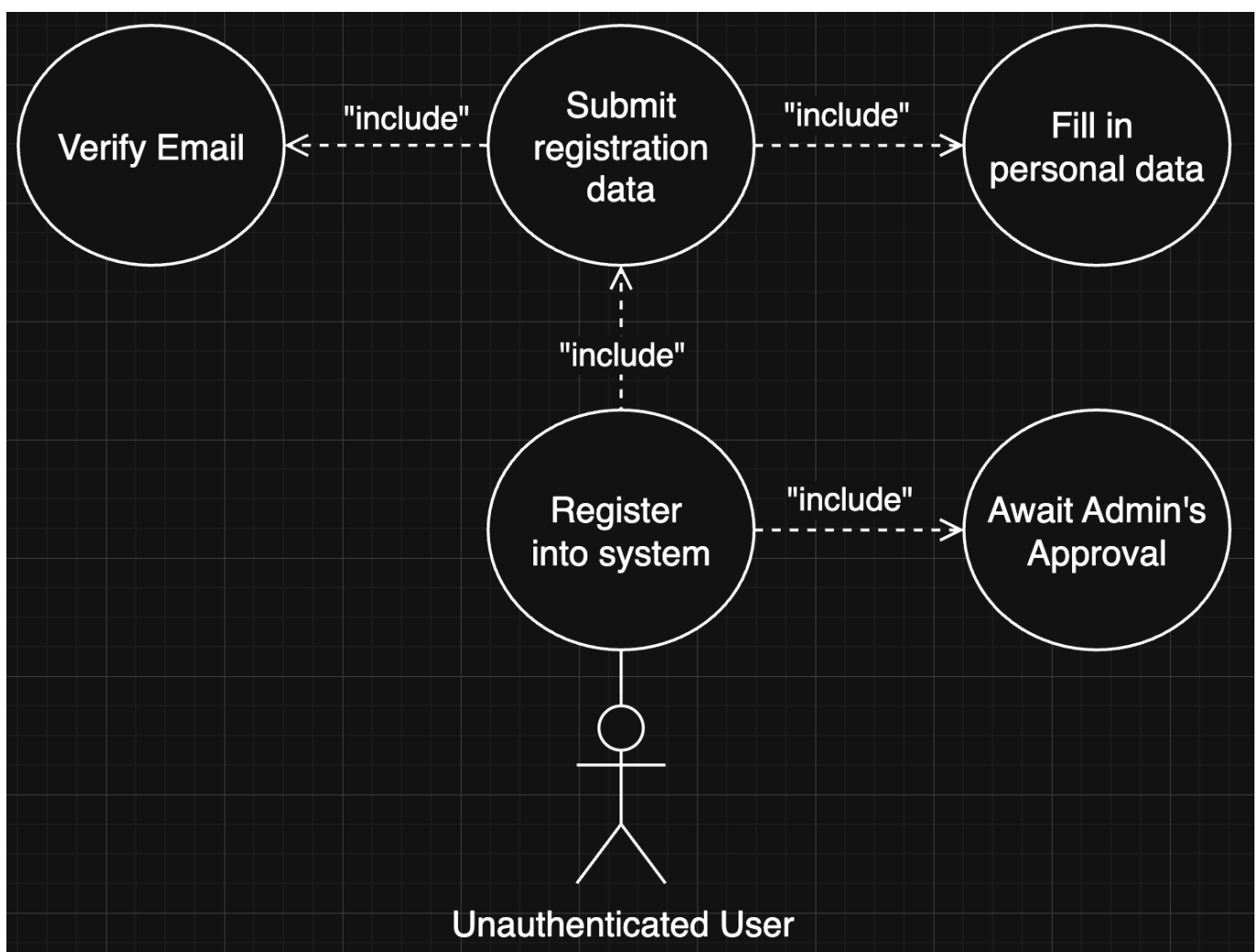


Figure 4.4 - UML Registration Use Case

2. **Document Creation:** The Figure 4.5 represents the process of creating a document within the system by an authenticated user. The actor begins by initiating document creation, which includes multiple dependent steps. The user must upload the document, after which the system checks whether it already exists on the blockchain. The process requires the creator's approval, the specification of participant IDs, and the subsequent approval of all tagged participants. Once these steps are completed, the creation is confirmed, finalizing the process and preparing the document for secure handling. The diagram emphasizes the structured and multi-party approval workflow that ensures document authenticity and prevents duplication.

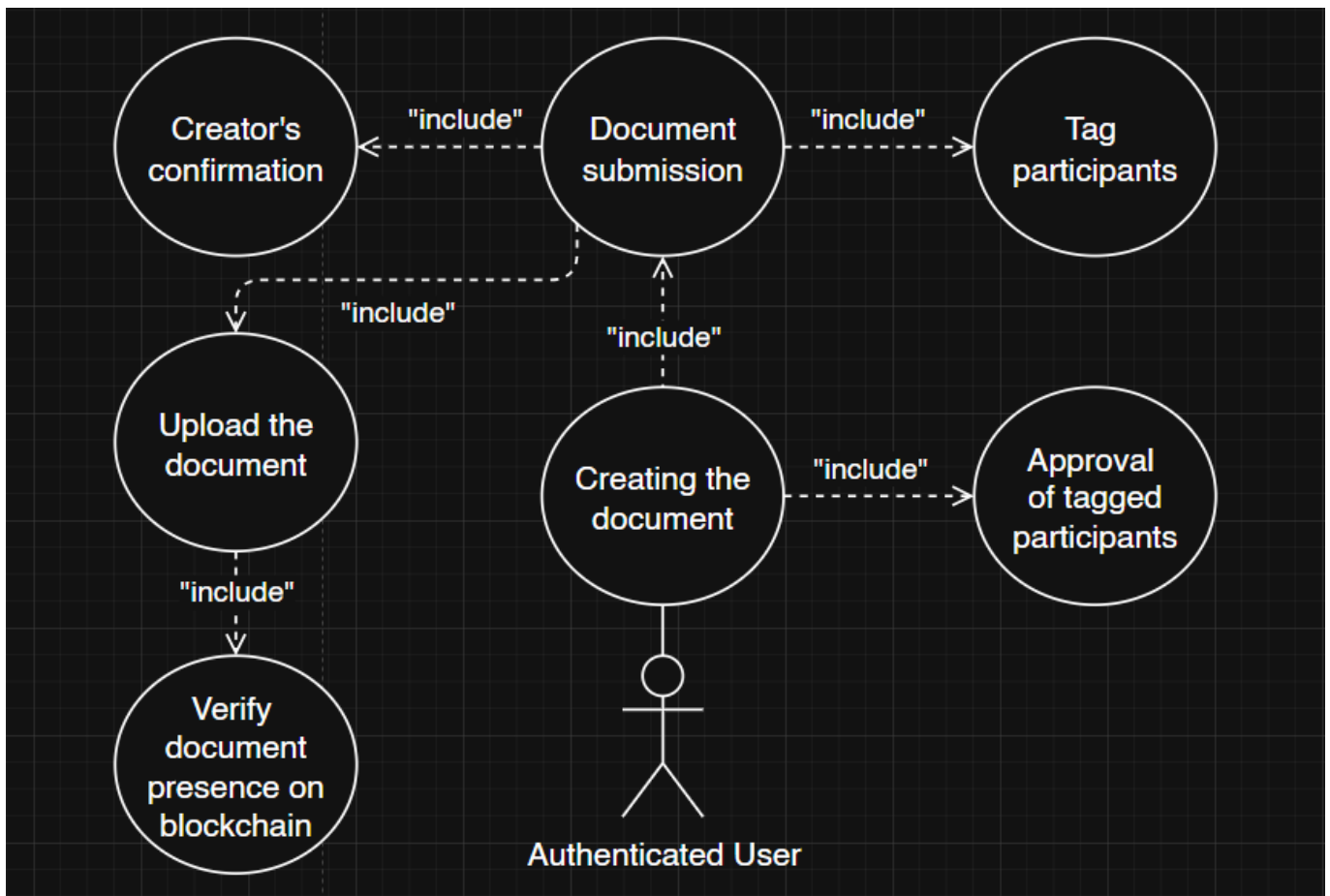


Figure 4.5 - UML Document Creation Use Case

3. **Document Verification:** The Figure 4.6 depicts the process of verifying a document by an authenticated user. The actor initiates the verification by uploading the document and submitting for verification. The system then calculates the cryptographic hash of the uploaded file and performs a check against existing blockchain records. If a corresponding record exists, the document is confirmed as authentic and unaltered; otherwise, the system indicates that no match exists. This diagram highlights the structured verification workflow, where hashing and blockchain lookups ensure document integrity and transparency in the verification process.

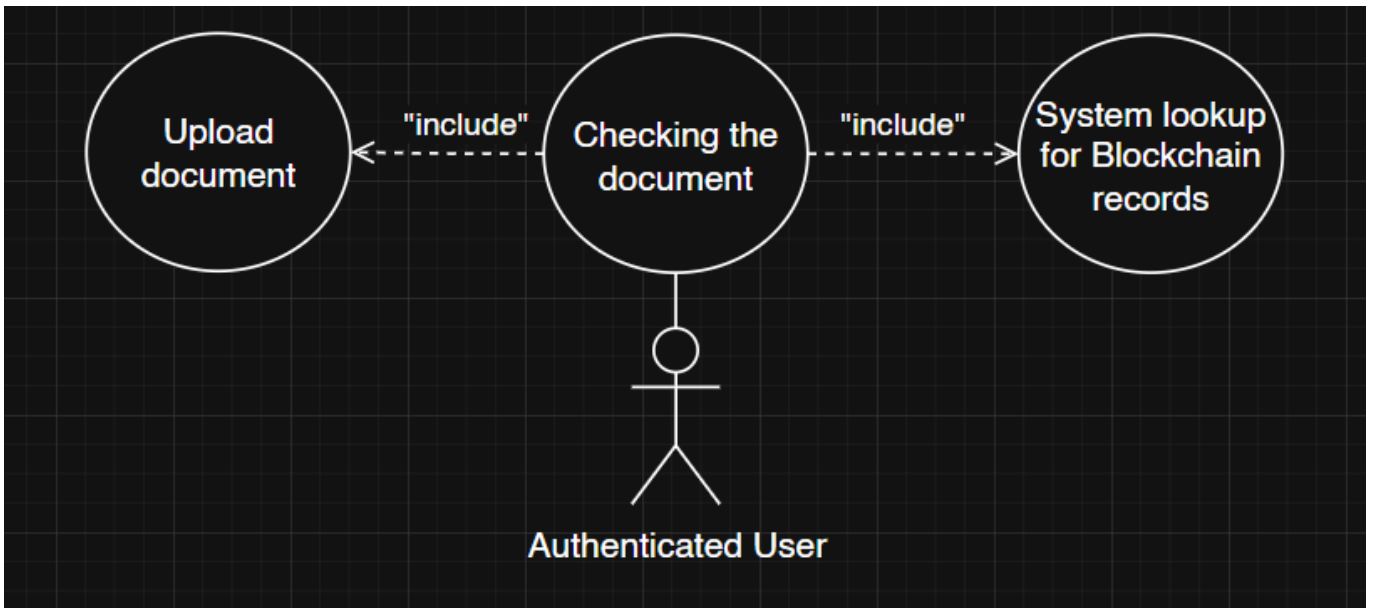


Figure 4.6 - UML Document Verification Use Case

4. **Account Approval:** The Figure 4.7 models the administrator's role in approving user registration. The administrator actor is responsible for validating new accounts by checking the uniqueness of the user and comparing the submitted registration data against records provided by the identity provider (IDP). The process also includes requesting personal data through the IDP to confirm authenticity before final approval. This ensures that only legitimate users, whose identities can be verified and who have not previously registered, are granted access to the system. The diagram emphasizes the administrator's critical role in safeguarding trust, preventing duplicate accounts, and maintaining the integrity of the registration process.

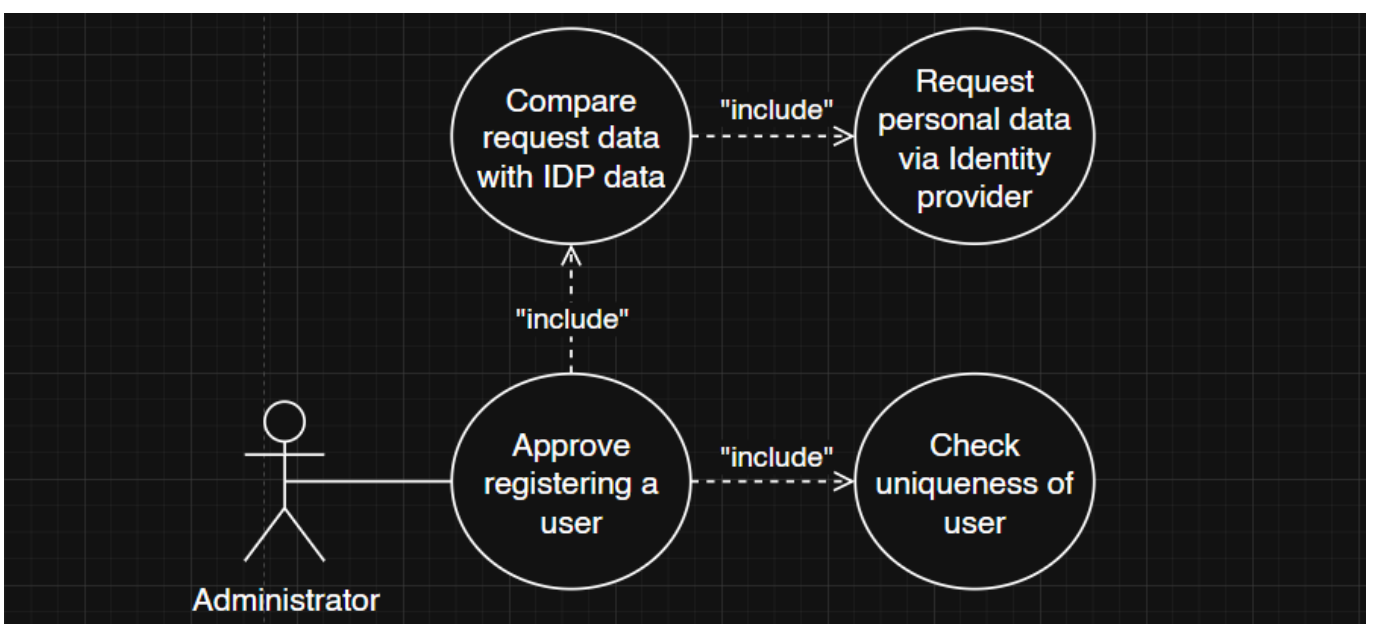


Figure 4.7 - UML Account Approval Use Case

4.2.2 Sequence Diagrams

The sequence diagrams illustrate the chronological flow of actions and messages for key scenarios, showing how the system components collaborate to achieve the intended operations.

1. **Registration process:** The Figure 4.8 illustrates the registration workflow for an unauthenticated user. The process begins when the user initiates registration, which triggers the client interface to start the registration procedure and confirm the initiation with the server. The client then renders the registration window where the user provides his email. At this point, the server initiates an email verification by sending a code to the user's email account. The client displays this code input field, and the user submits the verification code to confirm the email. The server validates the code and user can continue to introduce personal data. The user submits the full registration data, and the server saves the request, confirms the creation, and the client interface updates the UI to reflect successful registration request. This sequence diagram emphasizes the interaction between the user, client UI, server, and email service to ensure secure and validated registration.

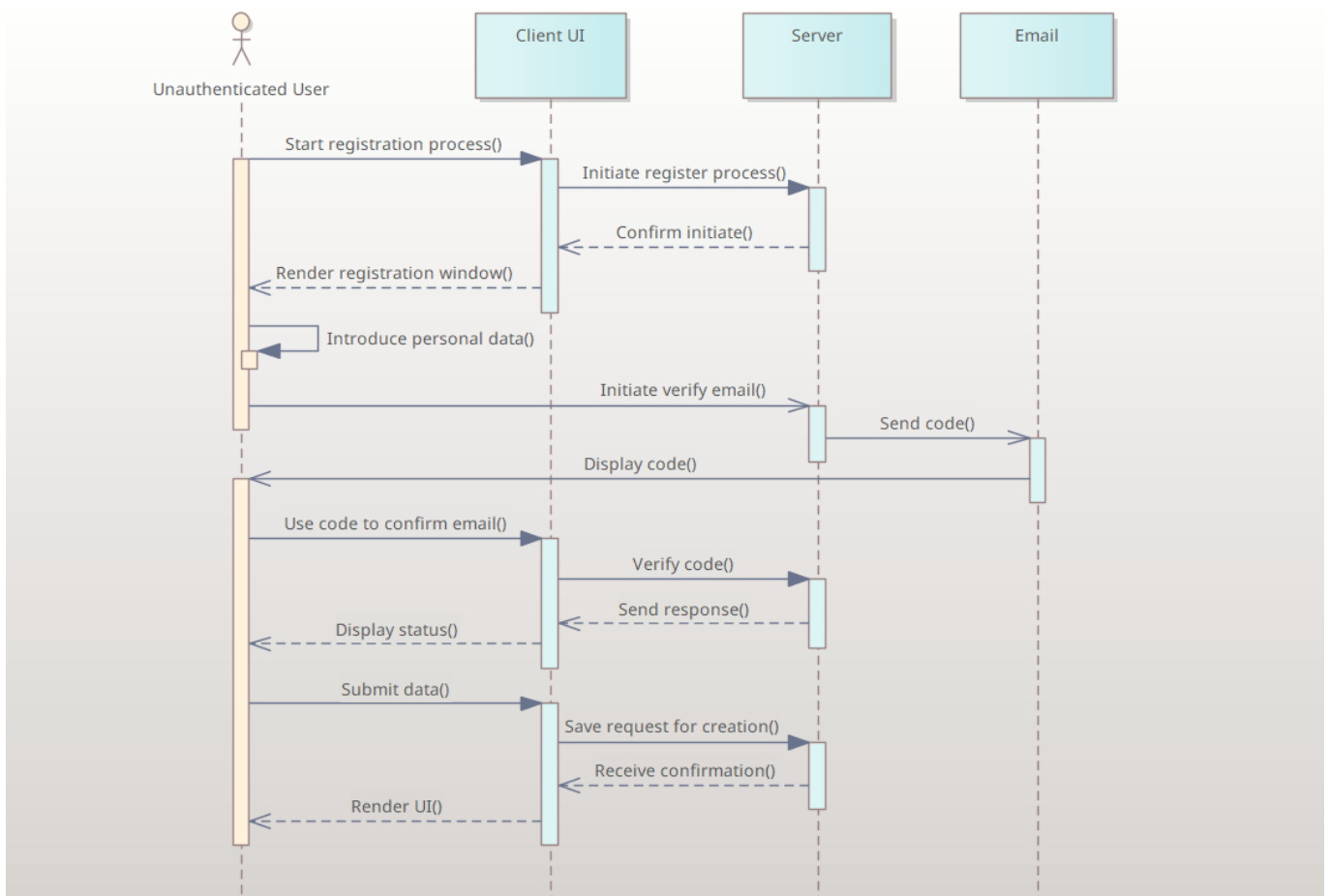


Figure 4.8 - UML Registration Sequence

2. **Document Creation:** The Figure 4.5 describes the process of document creation by an authorized user. The workflow starts when the user initiates the creation process through the client, which renders

the UI. The user uploads the document, provides the participants' IDs, and confirms the action. The client then sends a request for document creation to the server, which computes the hash of the file and checks the blockchain for existing records. If no duplicate is found, the server proceeds by distributing the document to all listed participants for approval. Once the approvals are collected, the server creates a new immutable record on the blockchain and confirms its creation. At the same time, a reference to the new document is stored in the database, linking it to the user's account. Finally, the confirmation is returned to the client, which updates the user interface with the new document's details. This sequence diagram highlights the coordinated interaction between client, server, blockchain, and database to ensure document integrity, approval, and traceability.

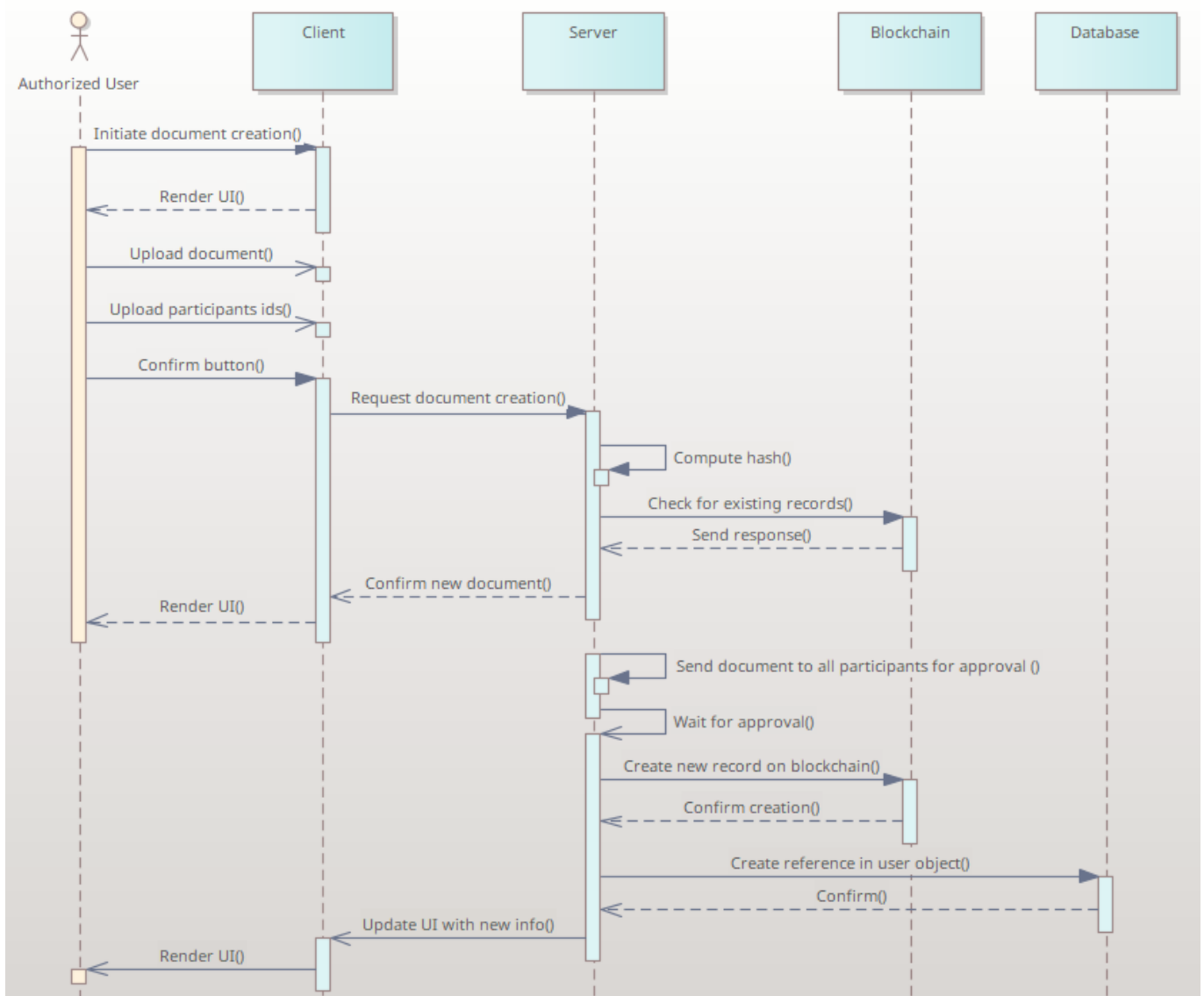


Figure 4.9 - UML Document Creation Sequence

- Document Verification:** The Figure 4.10 shows the workflow for document verification by an authorized user. The process begins when the user initiates the verification through the client interface,

which renders the UI. The user uploads the document and submits the form. The client then sends a verification request to the server. The server computes a cryptographic hash of the uploaded file and queries the blockchain to check whether a matching record already exists. The blockchain responds with the result of the lookup, which the server forwards back to the client. Finally, the client displays the verification outcome to the user. This diagram highlights the critical interaction between user actions, system components, and blockchain infrastructure to ensure document authenticity and integrity.

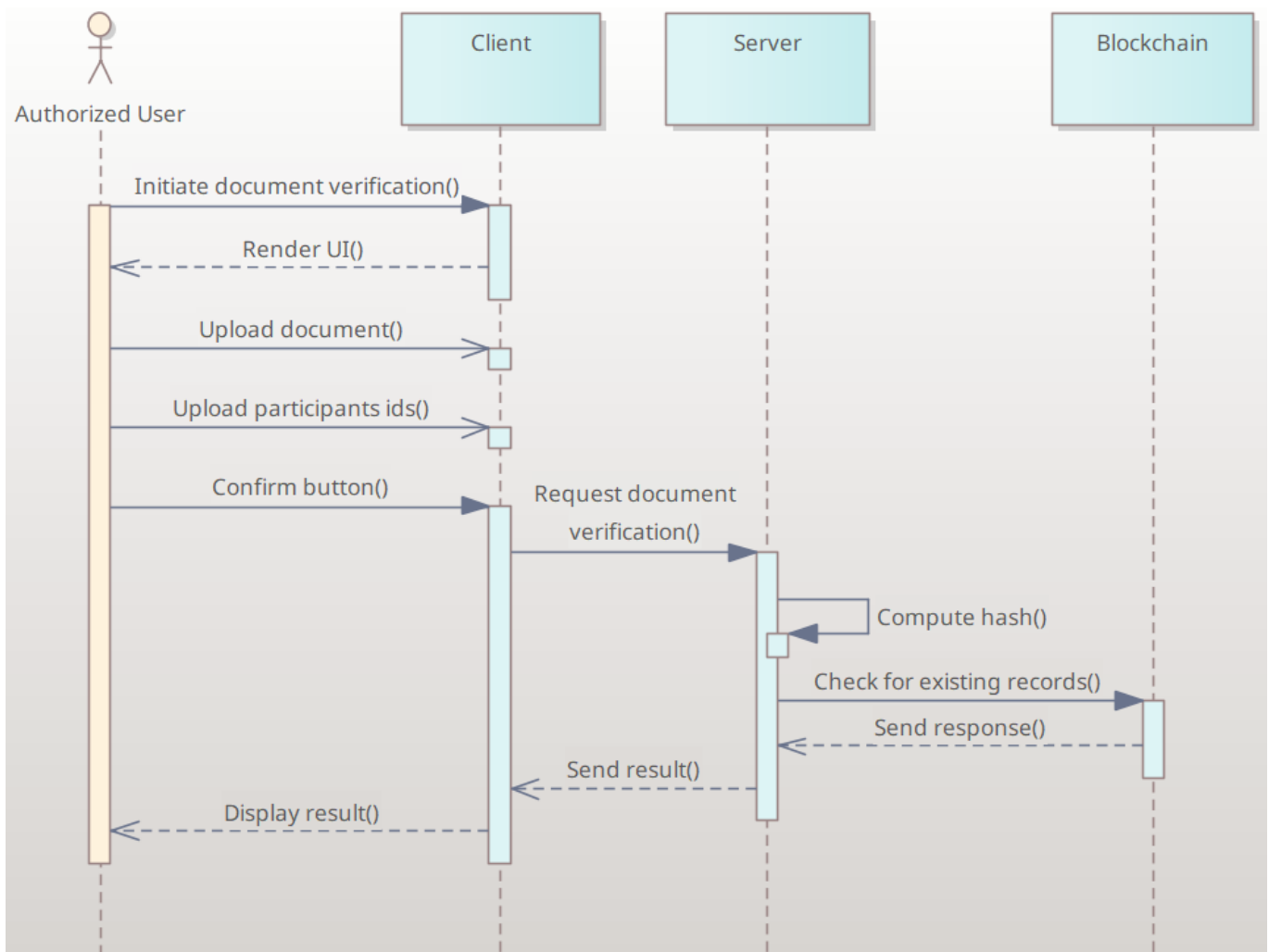


Figure 4.10 - UML Document Verification Sequence

4. **Account Approval:** The Figure 4.11 illustrates the user account approval process managed by the administrator. The flow begins when the administrator requests pending registration data for review, which the server provides to the admin panel. To validate the registration, the server queries the Identity Provider (IDP) for personal data associated with the submitted IDNP and returns the response. The administrator then compares the provided information against the IDP data. If the data is valid, the administrator approves the account creation. The server processes this approval by generating a new

user object in the database, after which a confirmation of account creation is returned to the administrator. This diagram highlights the administrator’s critical role in ensuring user identity verification and the coordinated interaction between the server, identity provider, and database during registration approval.

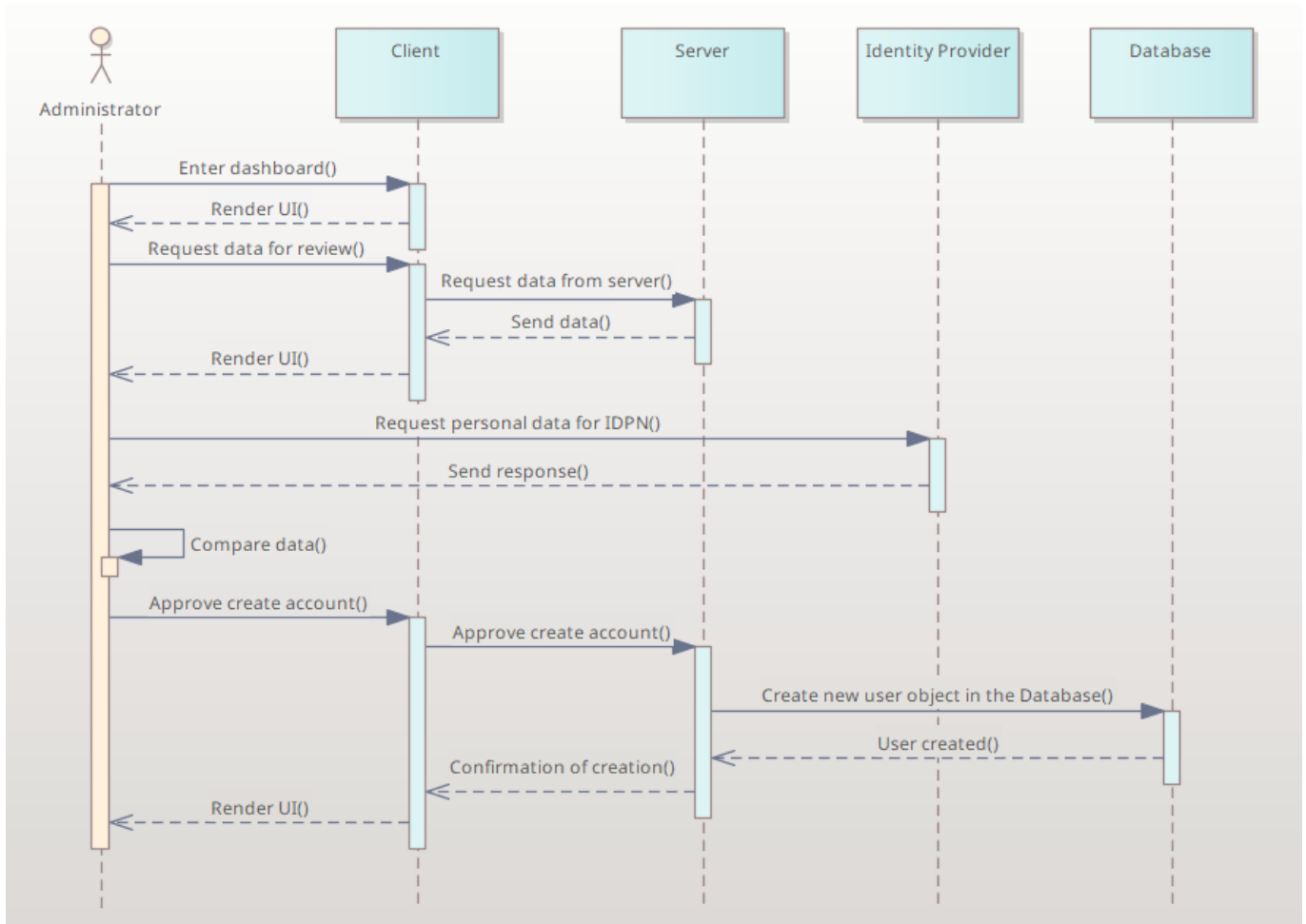


Figure 4.11 - UML Account Approval Sequence

4.3 Structural Modeling

This section presents the structural view of the system, focusing on the static aspects of its architecture. Structural modeling helps define the main components, their responsibilities, and the relationships between them.

4.3.1 Class Diagram

The Figure 4.12 models the core entities and relationships within the document notarization system. At the center is the User class, which contains attributes such as ID, full name, email, phone, status, role, and timestamps, alongside methods for document creation, signing, declining, and verification. Each user is assigned a Role, which defines permissions and enforces access control.

The Document class represents notarized files with properties like ID, title, owner, status, storage

reference, size, and creation details. Its methods manage the signing workflow, including adding participants, submitting for signing, and finalizing signatures. A document is linked to multiple Participants, each of whom has a role, status, and the ability to either sign or decline. These actions generate corresponding Signatures, which record details such as participant ID, signature type, reference, and timestamp.

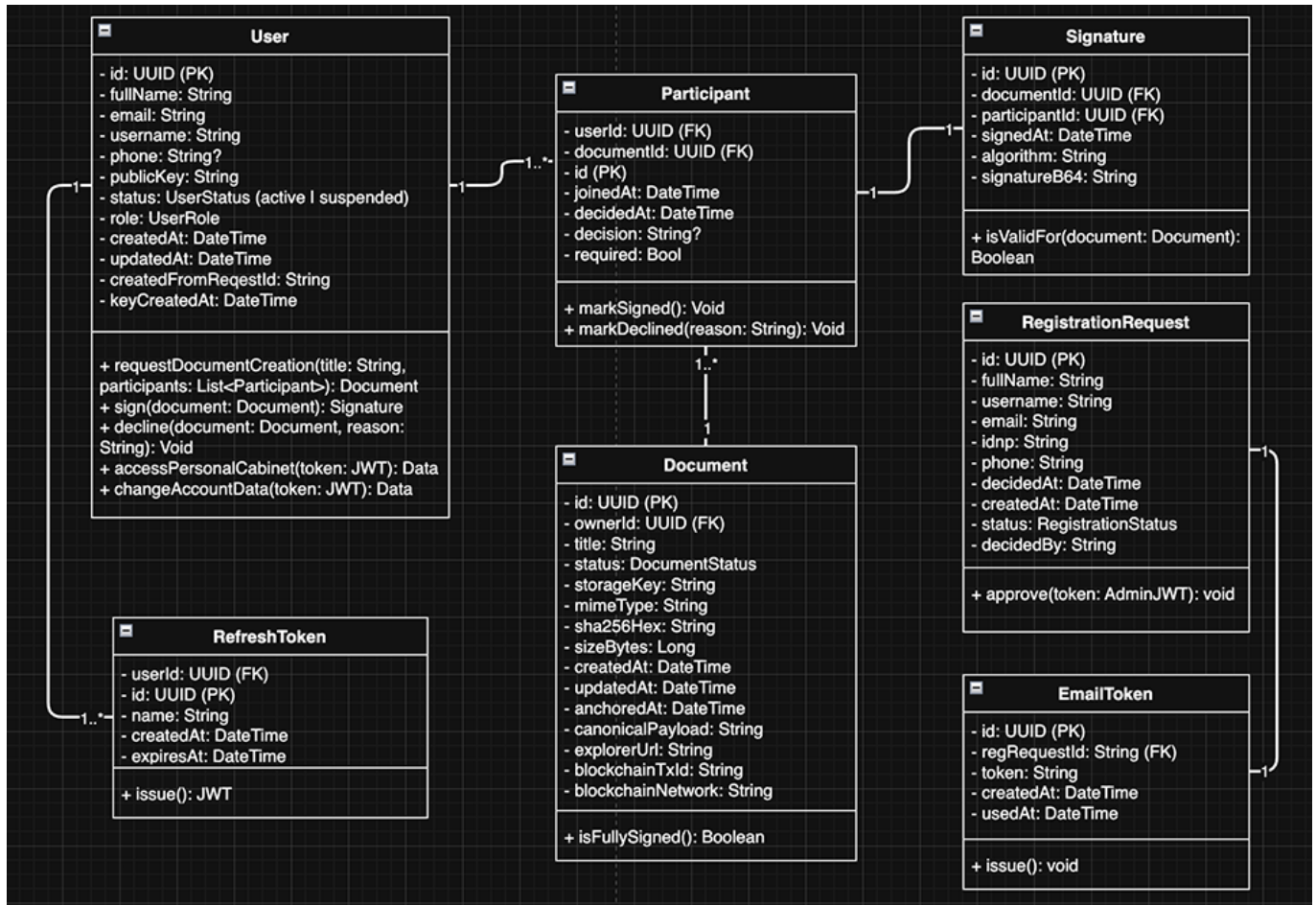


Figure 4.12 - UML Core Domain Class Model

Altogether, this diagram highlights how users, documents, participants, signatures, and registration requests interact. It reflects a cohesive domain model where documents are securely managed, signed by multiple parties, and validated against blockchain anchors to ensure integrity and trustworthiness.

4.4 Figma User Interface Mockups

The landing page (Figure 4.13) is the first page seen by the user when entering the site, so it gives the user a clear description of the functionality the service provides. In addition to the general introduction, the landing page highlights the main features of the platform such as secure digital document signing, document verification, and account management. The page also provides quick navigation buttons for login and registration, along with a concise call-to-action section that encourages new users to get started immediately. The design focuses on clarity, minimalism, and trust-building by including branding and possibly short explanations of how the service ensures security and authenticity.

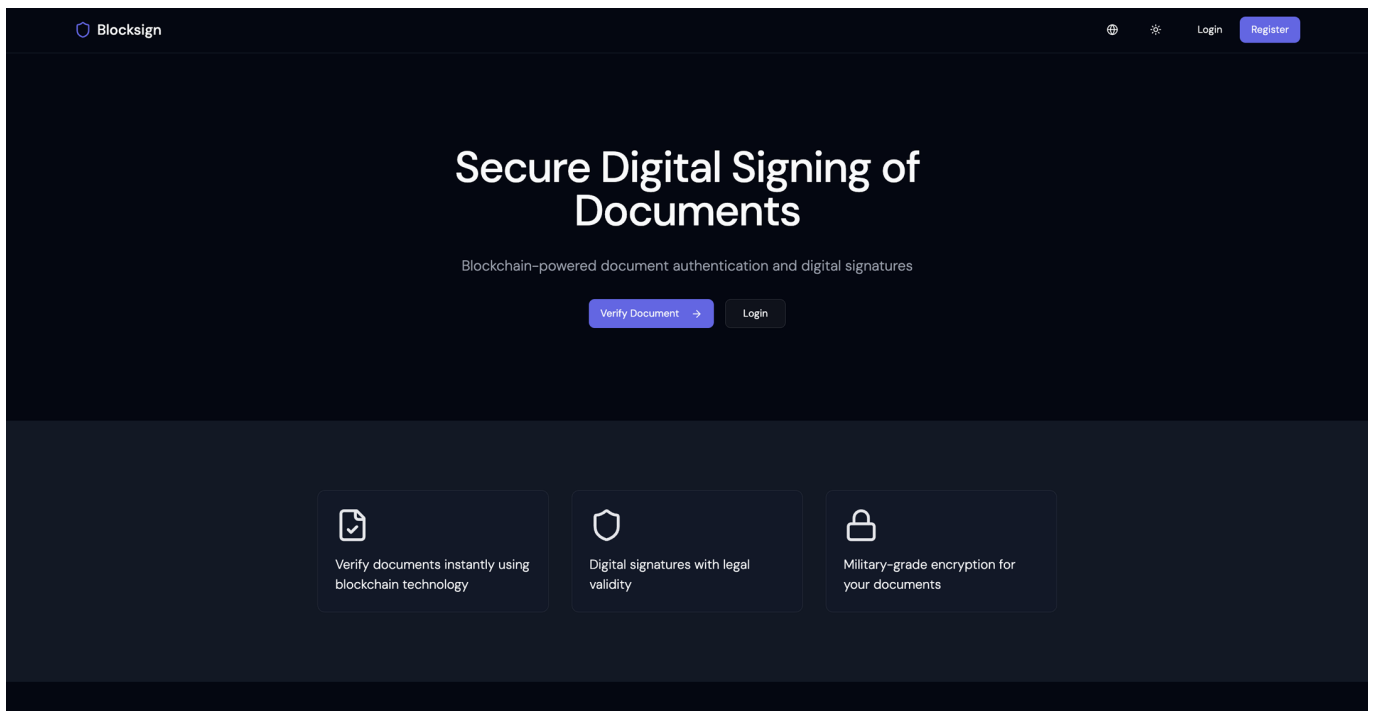


Figure 4.13 - Landing page

The login page (Figure 4.14) gives user the possibility of entering via email and seed phrase.

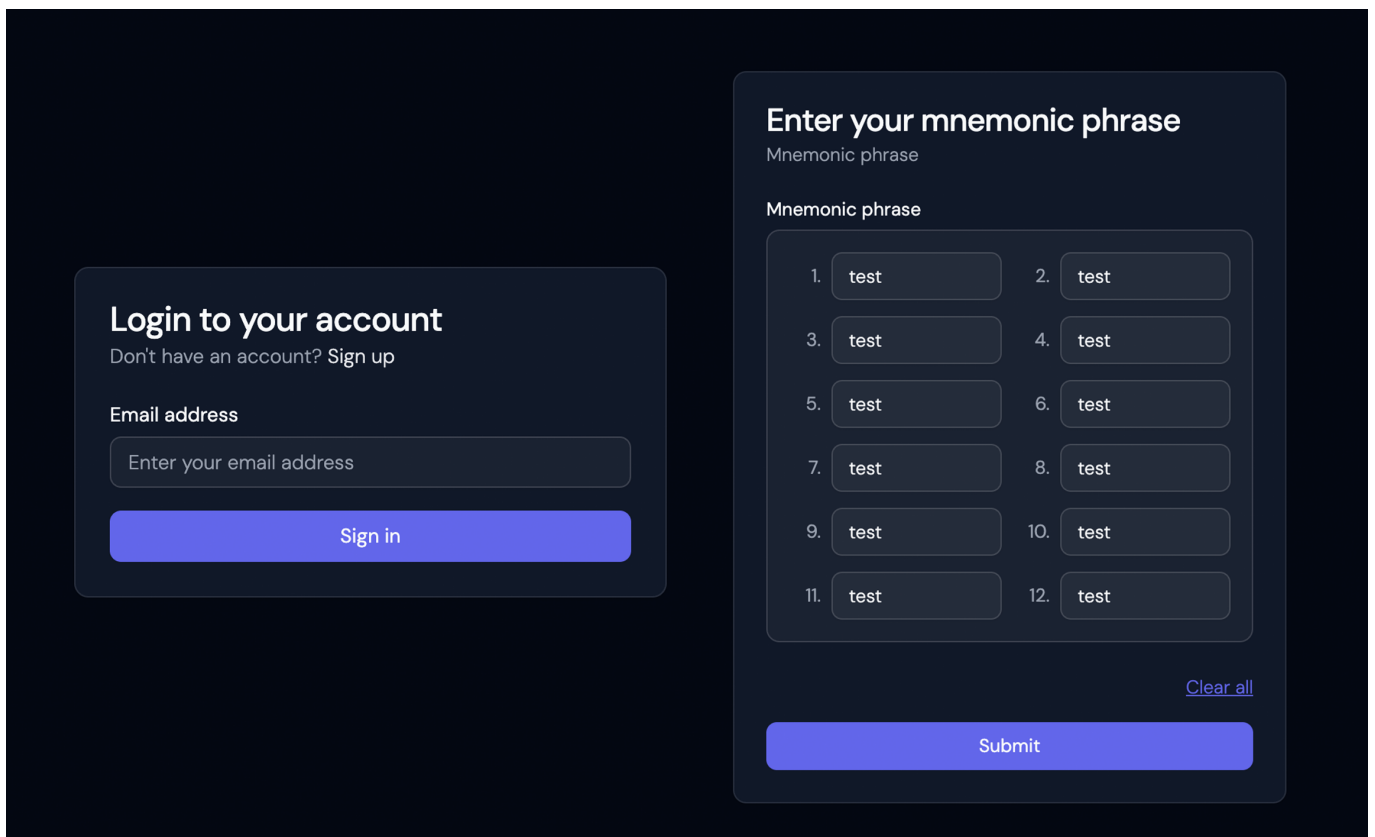


Figure 4.14 - Login page

The design maintains a simple, uncluttered layout to make the login process quick and intuitive. Se-

curity measures such as preventing brute-force attempts or notifying users of suspicious logins are included in the backend to protect user data.

The registration page (Figure 4.16) provides the user with the possibility to create an account in the service. The first step is to enter the email to which will be sent a verification code. This step ensures the validity of the email and protects against fake accounts. Without confirmation, the user cannot proceed to the next steps, which increases platform reliability and security.

Confirm the identity

Full Name
Enter your full name

Username
test_username1234

Phone
+1 123456789

IDNP
Enter your 13-digit IDNP

Birth Date
Pick a date

Selfie
Choose File No file chosen Select file

Accepts only jpeg, jpg, png, pdf, bmp, svg

The credentials are only used to verify the identity, then they are deleted

Confirm

Figure 4.15 - Register page - User info

The second step of registration (Figure 4.15) is submitting the form with personal data, which includes: full name, username, phone number, idnp, birth date and a selfie.

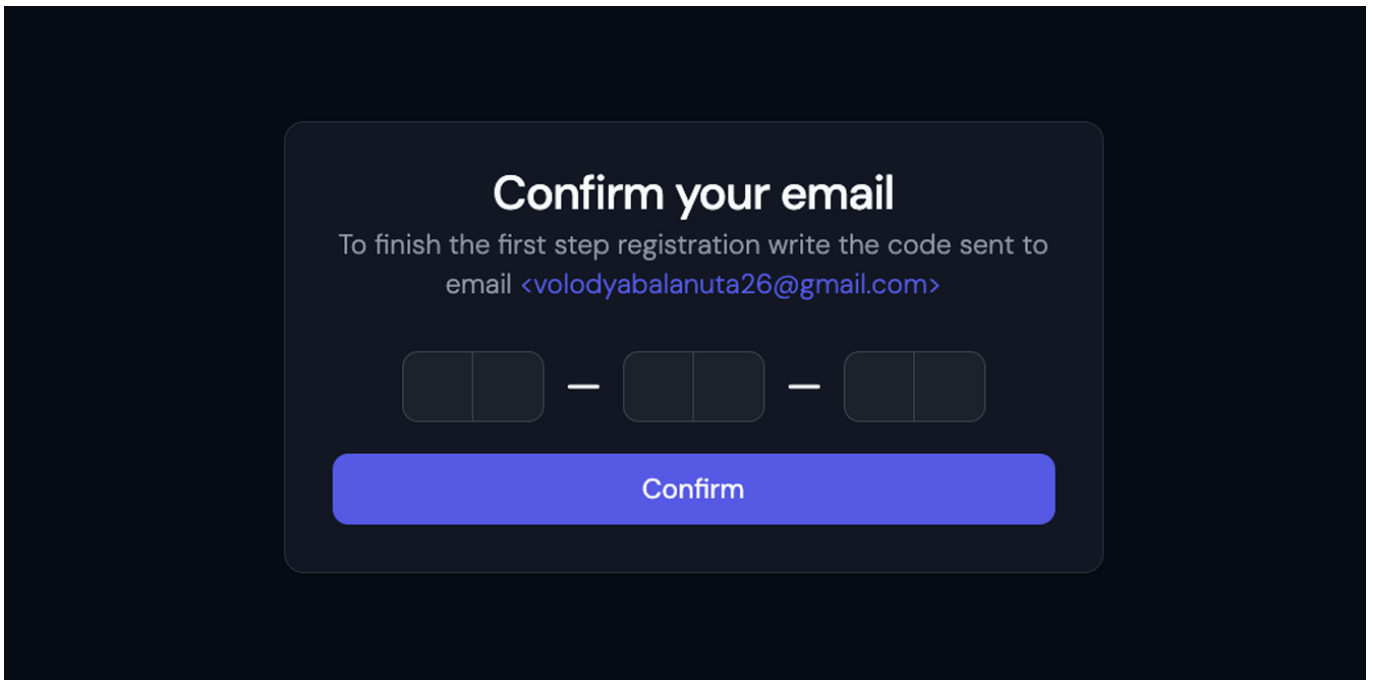


Figure 4.16 - Register - Verifying the email

After submitting the form the user sees the successful request (Figure 4.17) screen. This page confirms that the registration request was created and now user should await admin approval.

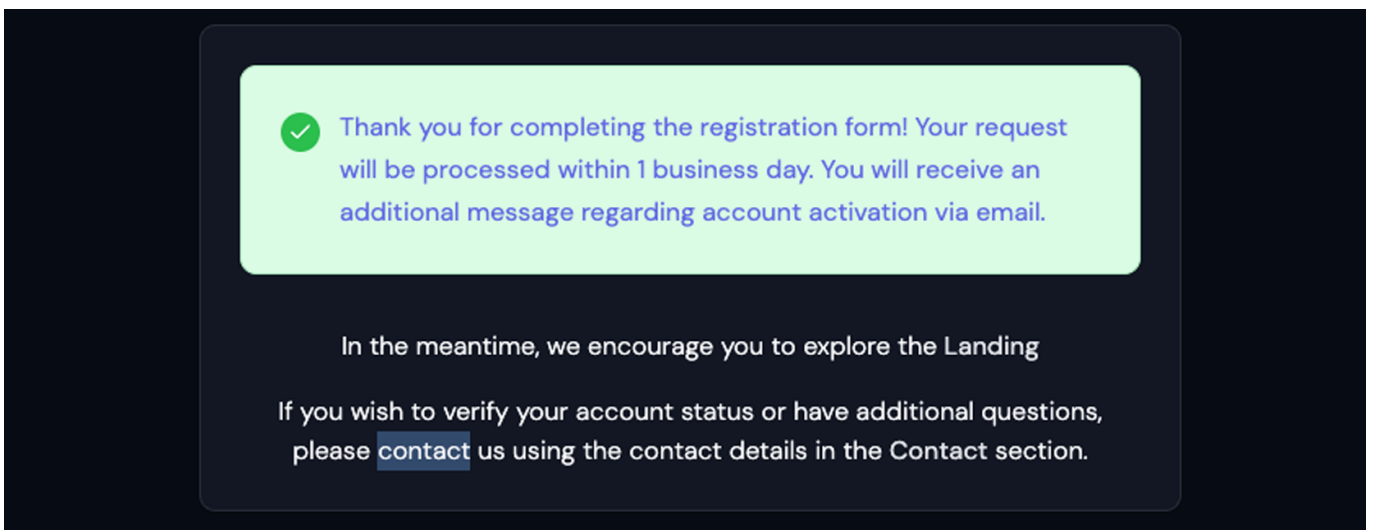


Figure 4.17 - Register - Successful finish

After confirmation, the user can finish registration process by following the link from the email and generating the seed phrase. Once this is done, the user gets access to the main features of the platform: documents (Figures 4.18), and account settings (Figures 4.22, 4.23).

The documents page has 2 main functionalities: verifying a document (Figure 4.18) and creating a new document (Figure 4.21). This structure makes the page a central hub for all document-related operations.

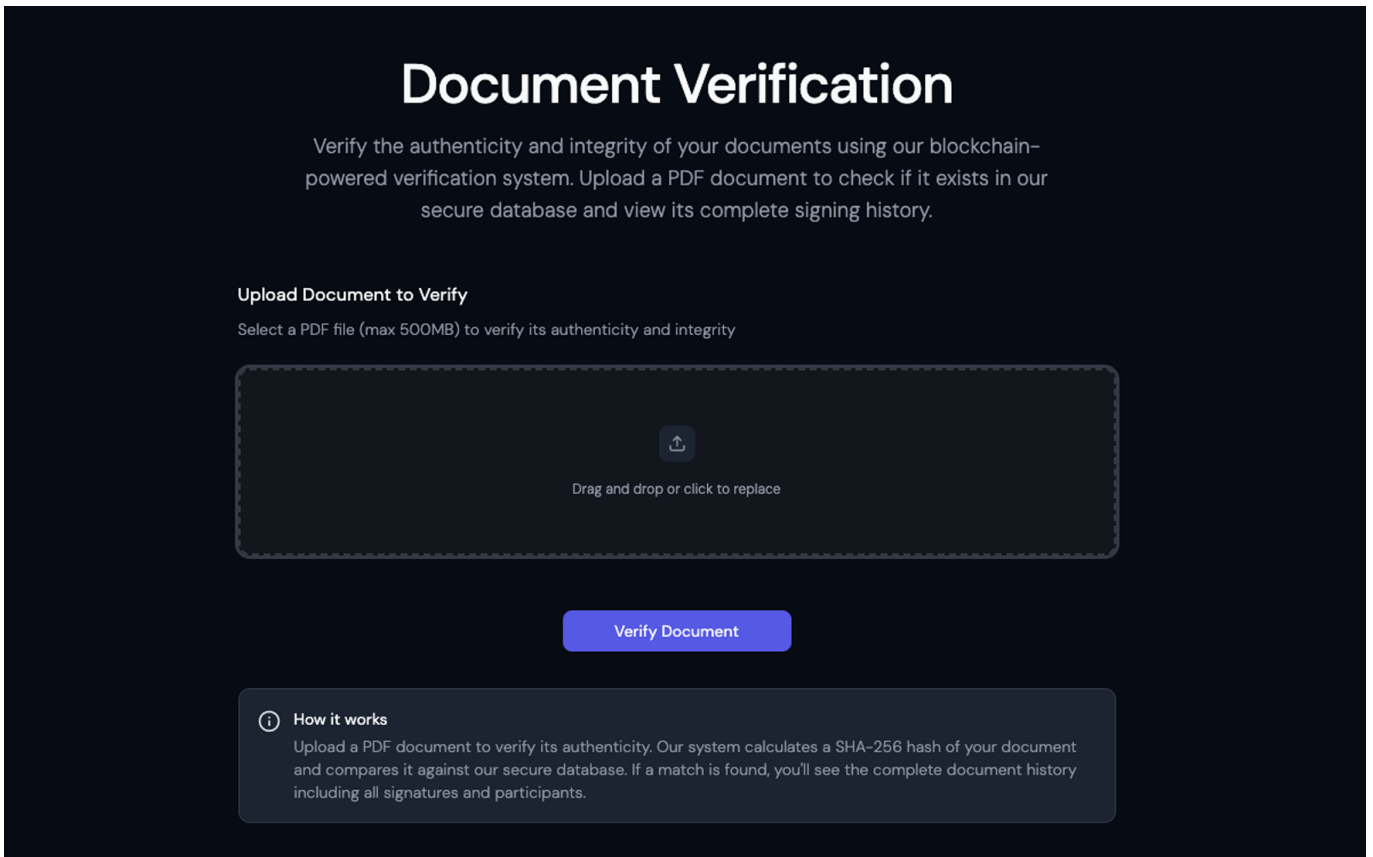


Figure 4.18 - Documents page - Verify an existing document

The results of verifying might be only “verified” (Figure 4.19) or “not found” (Figure 4.20). When a document is confirmed as existing, metadata such as the signing parties, date, document ID, owner are shown. If the document is not found, the system displays a message that there is no such document anchored to the blockchain.

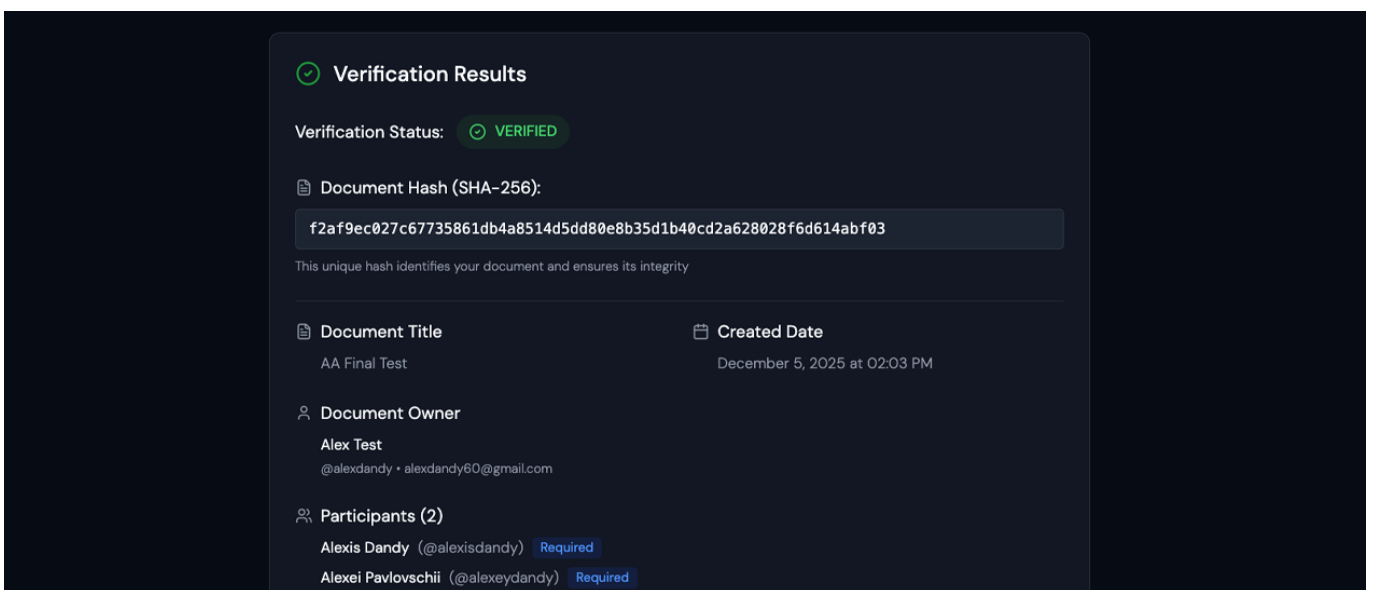


Figure 4.19 - Documents page - Existing instance of document

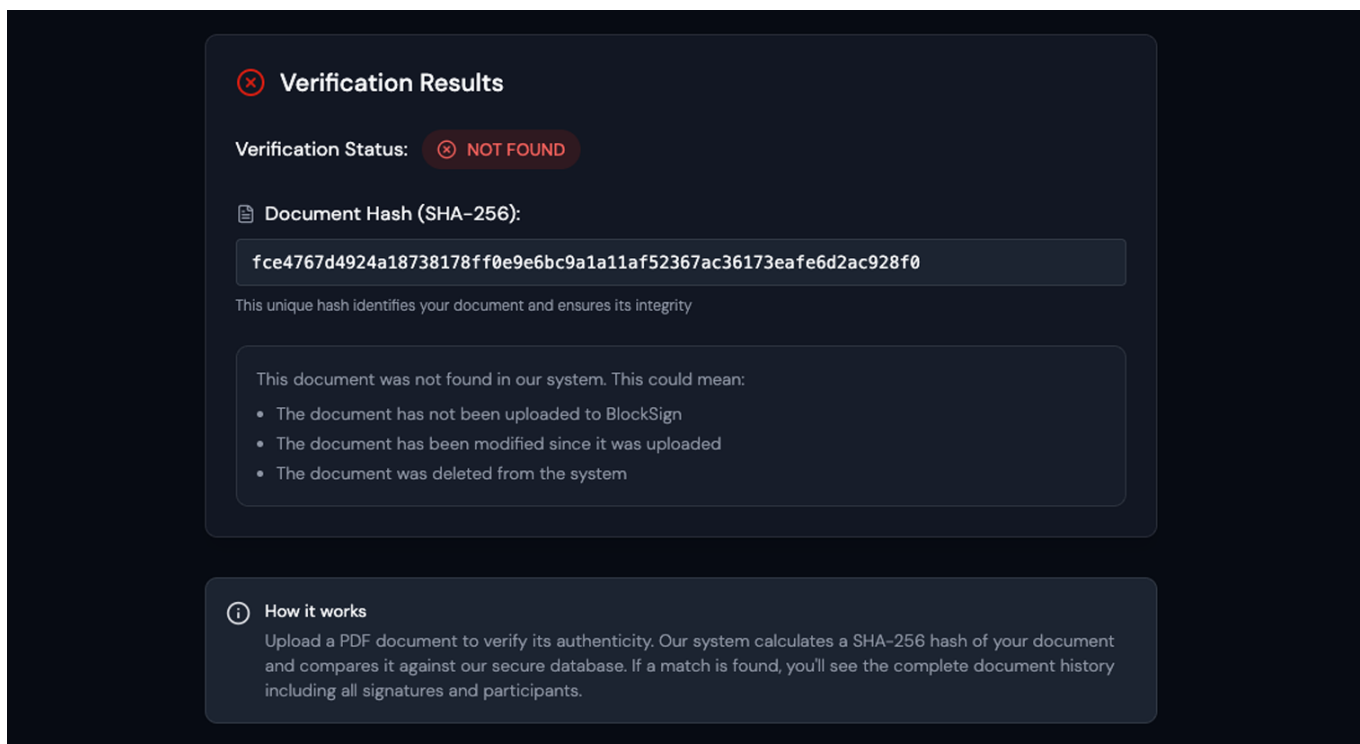


Figure 4.20 - Documents page - Not existing instance of document

On the document creation option (Figure 4.21) user can select a document and tag multiple parties. The system ensures that uploaded files are stored securely and encrypted until the signing process is complete.

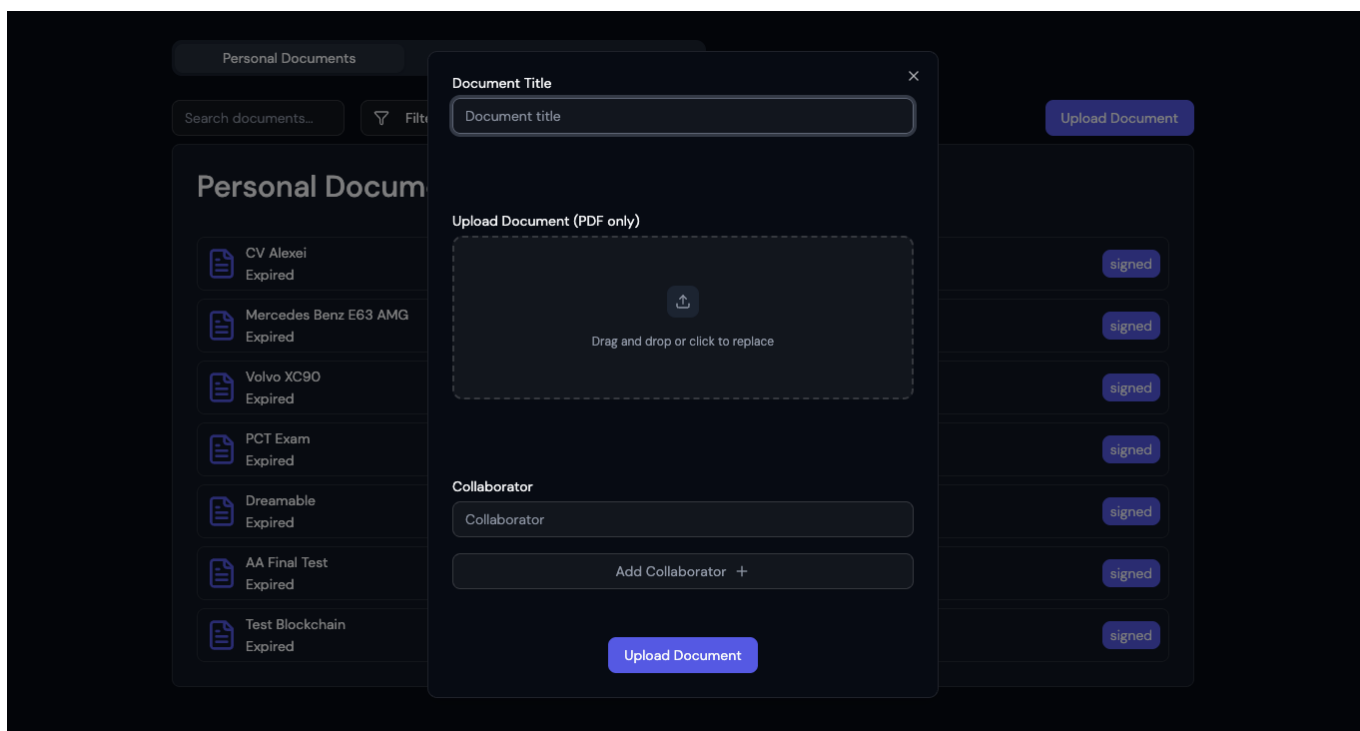


Figure 4.21 - Documents page - Upload and sign the document

The account page gives the ability to view the user details and set protection settings (Figure 4.22).

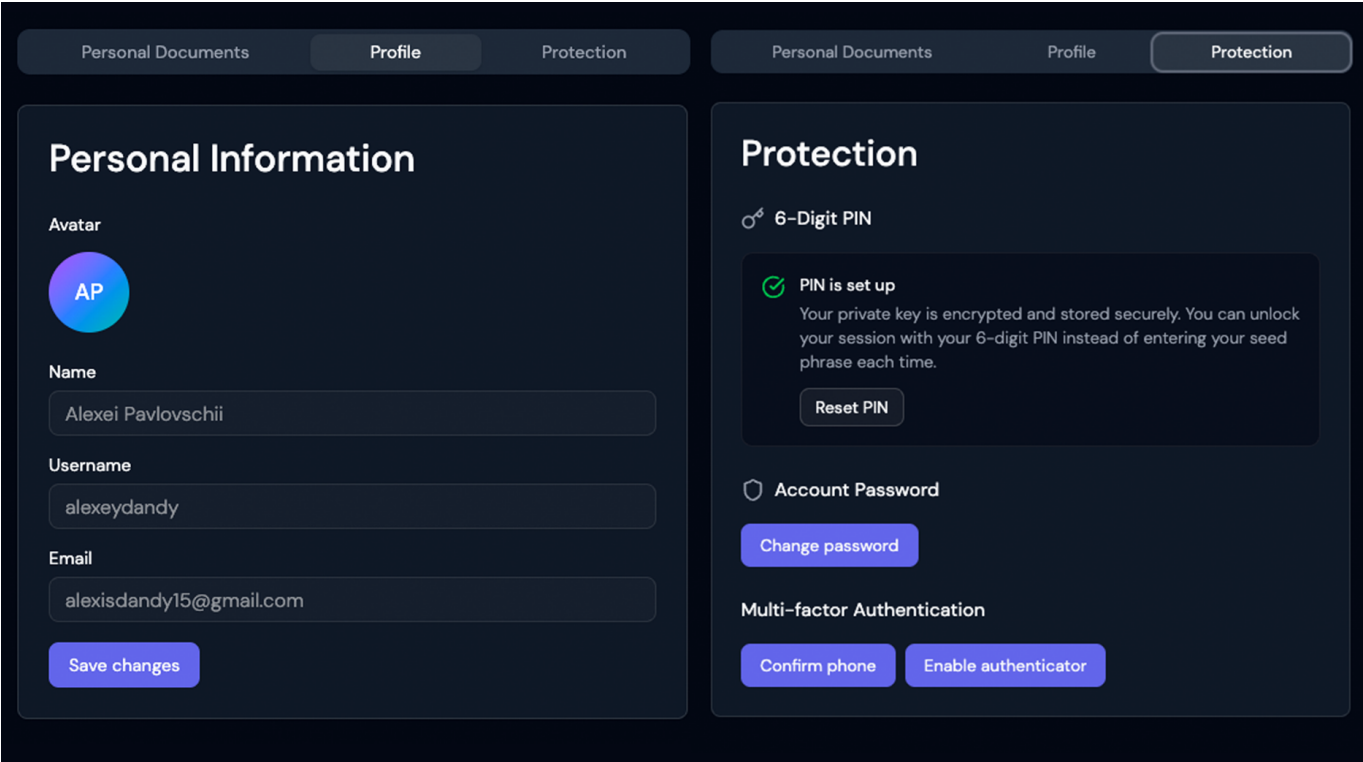


Figure 4.22 - Account page - Possibility to modify personal info and protection methods

The protection tab provides the possibility to change PIN, it may also include login history, trusted devices, and alerts for unauthorized access.

The documents tab (Figure 4.23) gives the user a list of their active and pending documents.

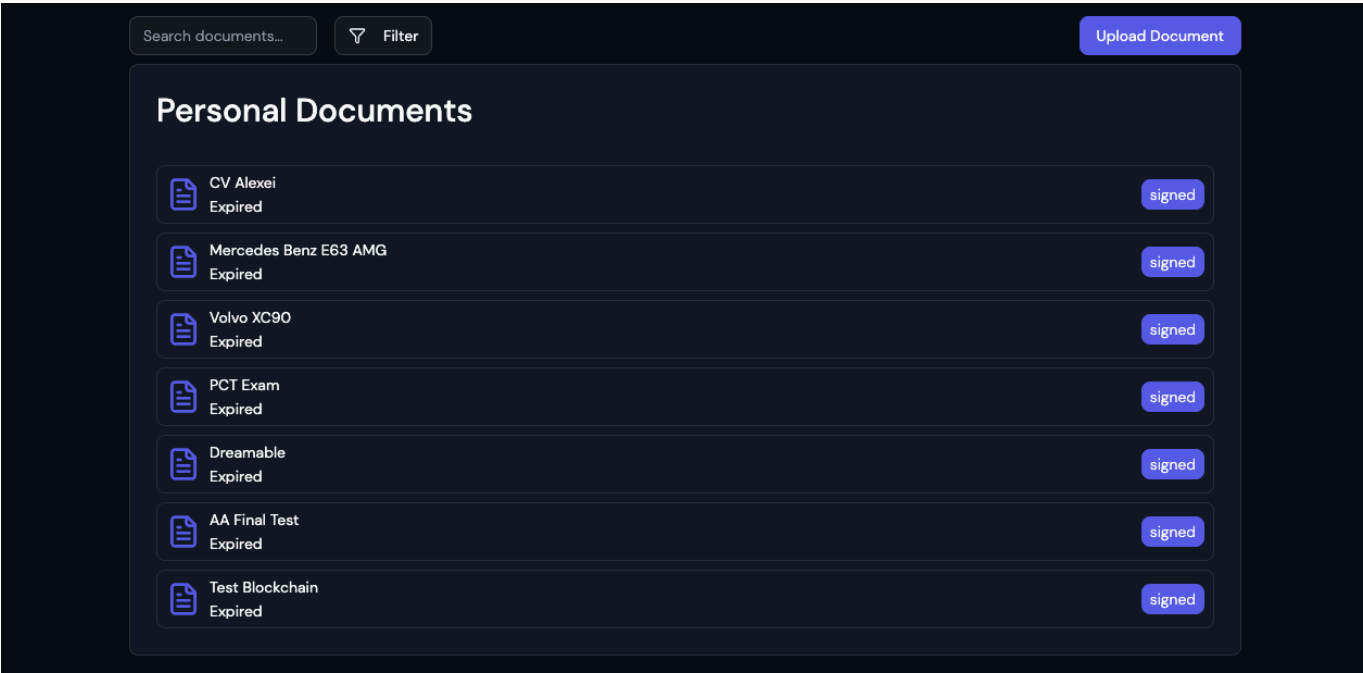


Figure 4.23 - Account page - View of the documents

5 Practical Implementation

The Project repositories are hosted on GitHub, are public and can be accessed by following these references: Back-end Core Logic[36], Front-end User Interface[37] and iOS Mobile[38].

5.1 Back-end architecture

The back-end of the system has been developed with a focus on scalability, modularity, and security. Its purpose is to provide all the critical functionality required for user registration, authentication, document management, and secure verification of signatures (Figure 5.1).



```
1 app.get('/api-docs.json', (_req, res) => res.json(openapiSpec));
2 app.get('/api/v1/health', generalLimiter, (_req, res) => res.json({ ok: true }));
3 app.use('/api/v1/auth', auth);
4 app.use('/api/v1/registration', registration);
5 app.use('/api/v1/documents', publicDocuments);
6 app.use('/api/v1/admin', requireAuth, admin);
7 app.use('/api/v1/admin/blockchain', requireAuth, blockchain);
8 app.use('/api/v1/user', requireAuth, user);
```

Figure 5.1 - Main Project Routers

The implementation relies on Node.js with the Express.js framework, while data persistence is handled by PostgreSQL through the Prisma ORM.

5.1.1 Application Layer

The server is structured around separate route modules, each of which is responsible for a specific part of the functionality. For example, the `auth.routes.ts` file manages the passwordless login process using challenge–response verification, while `registration.routes.ts` covers the steps of email OTP verification, registration requests, and account completion with a public key. The administrator’s responsibilities, such as reviewing and approving registration requests, are implemented in `admin.registration.routes.ts`. Finally, user-oriented functionality, including profile management and document operations, is centralized in `user.routes.ts`. This modular design improves maintainability and makes it easier to extend the system in the future.

5.1.2 Database Layer

The database schema is implemented using Prisma ORM on top of PostgreSQL. Entities such as User, RegistrationRequest, Document, DocumentParticipant and Signature are modeled explicitly with foreign keys and unique constraints to guarantee consistency (Figure 5.2).



```

1  model User {
2    id          String      @id @default(uuid())
3    email       String      @unique
4    fullName    String
5    username    String      @unique
6    publicKeyEd25519 String?
7    keyCreatedAt DateTime?
8    phone       String?     @db.VarChar(32)
9    role        UserRole    @default(USER)
10   status      UserStatus  @default(ACTIVE)
11   createdAt   DateTime    @default(now())
12   updatedAt   DateTime    @updatedAt
13
14   refreshTokens RefreshToken[] @relation("UserRefreshTokens")
15   documentsOwned Document[]   @relation("UserDocumentsOwned")
16   docParticipants DocumentParticipant[] @relation("UserDocumentParticipants")
17   signatures    Signature[]   @relation("UserSignatures")
18
19   createdFromRequestId String?
20 }

```

Figure 5.2 - User Database Model

The use of Prisma Client enables type-safe queries and simplifies operations like migrations, seeding, and testing.

5.1.3 Security Measures

The architecture incorporates several security mechanisms. Authentication challenges are valid only for five minutes and become invalid once used. Refresh tokens are stored in secure cookies to reduce exposure. The use of Ed25519 ensures modern cryptographic security, while SHA-256 guarantees the integrity of documents. Finally, file validation ensures that only PDF documents are accepted.

5.1.4 Authentication and Tokens

Authentication in the system is fully passwordless and relies on public/private key pairs. Users prove their identity by signing challenges provided by the server. Successful verification grants them access tokens and refresh tokens in the form of JWTs.

Access tokens are short-lived and returned in the response body, while refresh tokens are long-lived and stored securely in HTTP-only cookies. Middleware functions such as `requireUser` and `requireAdmin` check the validity of tokens and enforce role-based access control.

5.1.5 Email Service

The system integrates Nodemailer as an email delivery mechanism configured with SMTP credentials for the project's domain. Emails are used in two important contexts. First, during registration, users receive a six-digit OTP code for email validation. Second, during document workflows, participants receive notifications when they are asked to review and sign a document, and again when the document has been finalized (Figure 5.3).

A code editor window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. It contains a JavaScript code snippet for sending an email using Nodemailer. The code is as follows:

```
1 export async function sendEmail(to: string, subject: string, html: string, extra?: Partial<nodemailer.SendMailOptions>) {
2   const info = await transporter.sendMail({
3     from: MAIL_FROM || `BlockSign <${SMTP_USER}>`,
4     to,
5     subject,
6     html,
7     ...extra
8   });
9   console.log('[mailer] sent:', info.messageId);
10 }
```

Figure 5.3 - Email Sending Mechanism

5.1.6 Cryptographic Module

Security of authentication and documents is achieved with the Ed25519 digital signature scheme, implemented via the @noble/ed25519 library (Figure 5.4).

A code editor window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. It contains a JavaScript code snippet for generating key pairs using the @noble/ed25519 library. The code is as follows:

```
1 const mnemonic = generateMnemonic(wordlist, 128); // 12 words (128 bits).
2 const passphrase = ''; // optional user-provided extra password (BIP-39 optional)
3
4 const seed = mnemonicToSeedSync(mnemonic, passphrase); // Buffer/Uint8Array
5
6 // Derive Ed25519 SLIP-0010 key at your fixed path
7 const PATH = "m/44'/53550'/0'/0'/0'"; // BlockSign path
8 const { key: privateKey } = derivePath(PATH, Buffer.from(seed).toString('hex'));
9
10 const publicKey = await ed.getPublicKeyAsync(privateKey);
```

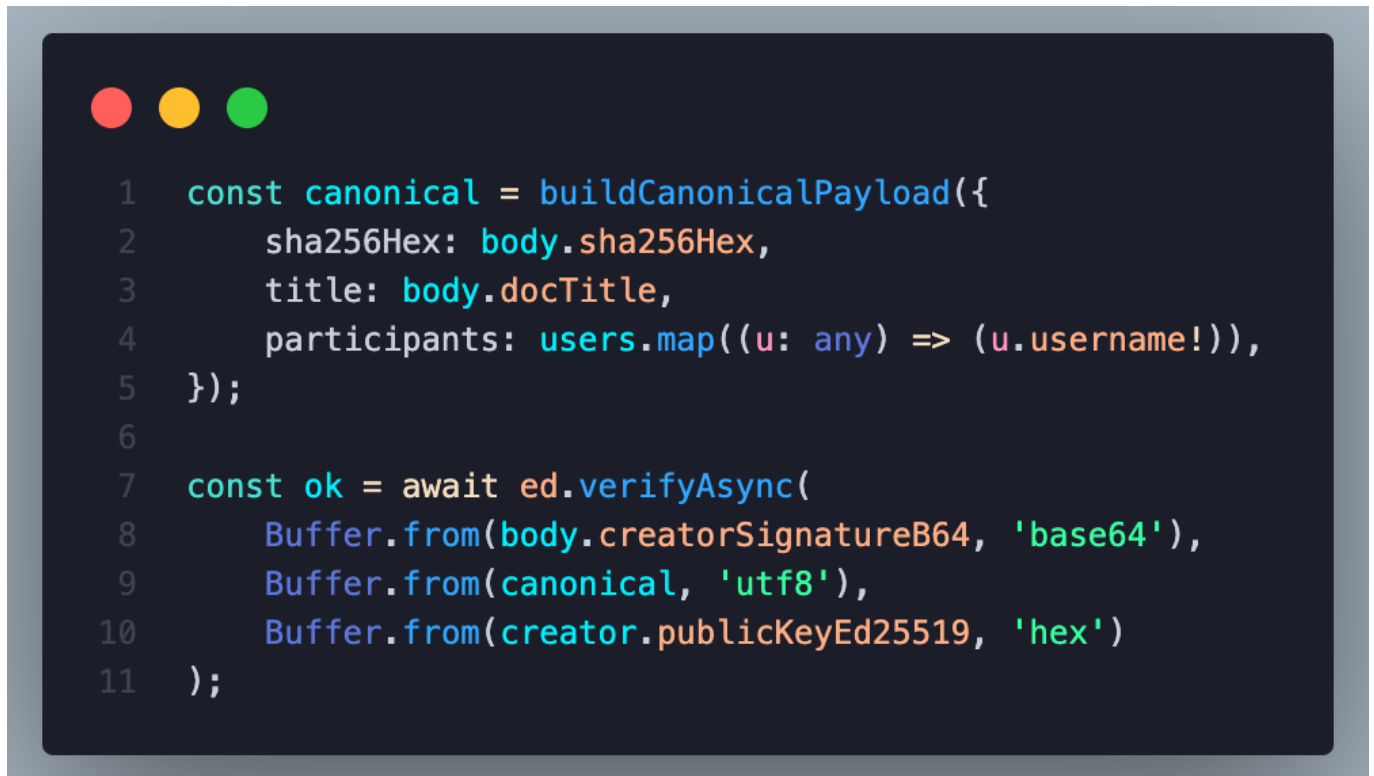
Figure 5.4 - Key Pairs Generation Mechanism

A dedicated crypto utility handles key pair generation, signing, and verification of messages. In addition, SHA-256 is used for hashing documents, ensuring their integrity before they are accepted or signed.

5.1.7 Document Workflow

The MVP implementation of the document signing process follows a well-defined lifecycle. When a user creates a document, they upload the file, calculate its SHA-256 hash, and sign a canonical payload. The payload and initial signature are stored in the database.

Participants selected by the creator receive an email with the PDF attached and the corresponding hash. Each participant is expected to calculate the hash independently, compare it with the payload, and if it matches, sign the document (Figure 5.5).



```
1  const canonical = buildCanonicalPayload({
2    sha256Hex: body.sha256Hex,
3    title: body.docTitle,
4    participants: users.map((u: any) => (u.username!)),
5  });
6
7  const ok = await ed.verifyAsync(
8    Buffer.from(body.creatorSignatureB64, 'base64'),
9    Buffer.from(canonical, 'utf8'),
10   Buffer.from(creator.publicKeyEd25519, 'hex')
11  );
```

Figure 5.5 - Build Payload and Verify Signature

Their signatures are then added to the database. Once all required signatures are collected, the document status changes to signed, and a notification is sent to all involved parties.

5.1.8 Extensibility

The modular design of the back-end makes it adaptable to future improvements. Currently, the project works on Polygon blockchain, but there can be added the functionality of choosing the blockchain network by the document creator. Similarly, while usernames are currently used to tag participants, the design leaves room for later integration with decentralized identity systems or phone book integrations.

5.2 Front-end UI

The front-end of the BlockSign system represents a modern, secure, and user-centric web application built using cutting-edge technologies. The implementation emphasizes client-side security, responsive

design, and intuitive user experience while maintaining the highest standards of cryptographic security.

5.2.1 Frontend Architecture

The front-end is built using Next.js 14 with React.js as the core framework, leveraging TypeScript for type safety and enhanced developer experience. The application utilizes Tailwind CSS for styling, providing a consistent and responsive design system across all components (Figure 5.6).



Figure 5.6 - Frontend Technology Stack Implementation

The architecture follows a modular component-based approach with clear separation of concerns:

- **Presentation Layer:** React components with Tailwind CSS styling
- **State Management:** React Context API for global state management
- **Security Layer:** Client-side cryptographic operations and secure key management
- **Communication Layer:** HTTP clients for API communication with the backend
- **Internationalization:** Multi-language support with i18next

The front-end implementation prioritizes client-side security, ensuring that sensitive information such as private keys and mnemonic phrases never leave the user's browser. This approach aligns with the zero-trust security model and provides users with complete control over their cryptographic materials.

5.2.2 Landing Page and User Onboarding

The landing page serves as the primary entry point for users, featuring a clean and professional design that communicates the platform's value proposition effectively. The page is structured to provide

comprehensive information about BlockSign's capabilities while maintaining an elegant and accessible design.

Landing Page Design and Layout

The landing page follows modern web design principles with a focus on clarity, accessibility, and conversion optimization. The page is divided into several key sections that progressively introduce users to the platform's features and benefits.

The header section features the BlockSign logo and navigation menu, providing immediate access to registration and login functionality. The hero section contains a compelling value proposition with clear call-to-action buttons that guide users toward registration or learning more about the platform.

The design emphasizes trust and security through the use of professional typography, consistent color schemes, and carefully selected imagery that reinforces the platform's reliability and sophistication.

Feature Presentation

The landing page includes dedicated sections that highlight BlockSign's key features:

- **Passwordless Authentication:** Emphasis on modern security through cryptographic keys
- **Document Integrity:** Explanation of cryptographic verification and tamper-proof signatures
- **Multi-party Signing:** Collaborative document workflows with multiple participants
- **Regulatory Compliance:** Adherence to international digital signature standards
- **User Privacy:** Client-side cryptography and zero-trust architecture

Educational Content

The landing page serves an educational purpose by explaining digital signature concepts to users who may be unfamiliar with cryptographic authentication. This includes simplified explanations of:

- How digital signatures work and why they're more secure than traditional methods
- The benefits of passwordless authentication using mnemonic phrases
- Document integrity verification and tamper detection
- Legal validity and compliance aspects of digital signatures

Trust Indicators

The page incorporates various trust indicators to build user confidence:

- Security certifications and compliance badges
- Transparent explanation of the platform's security architecture
- Clear privacy policy and data handling practices
- Educational resources about digital signature technology

The user onboarding process is designed to be intuitive and secure, guiding users through each step of the registration and setup process with clear instructions and visual feedback. Progressive disclosure techniques are used to avoid overwhelming new users while ensuring they understand the security implica-

tions of their actions.

5.2.3 User role specified UI

The authorized user is represented by the possibility to view his account info in the header as well as having access to the authorized only pages (Figure 5.7).

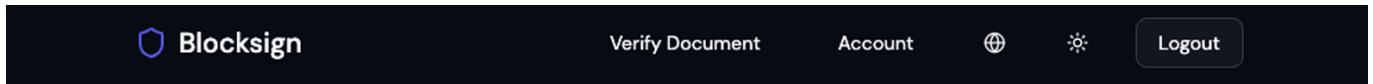


Figure 5.7 - Header Account User

Navigating to the admin console will lead to the page, where admin can confirm or reject the registration requests (Figure 5.8).

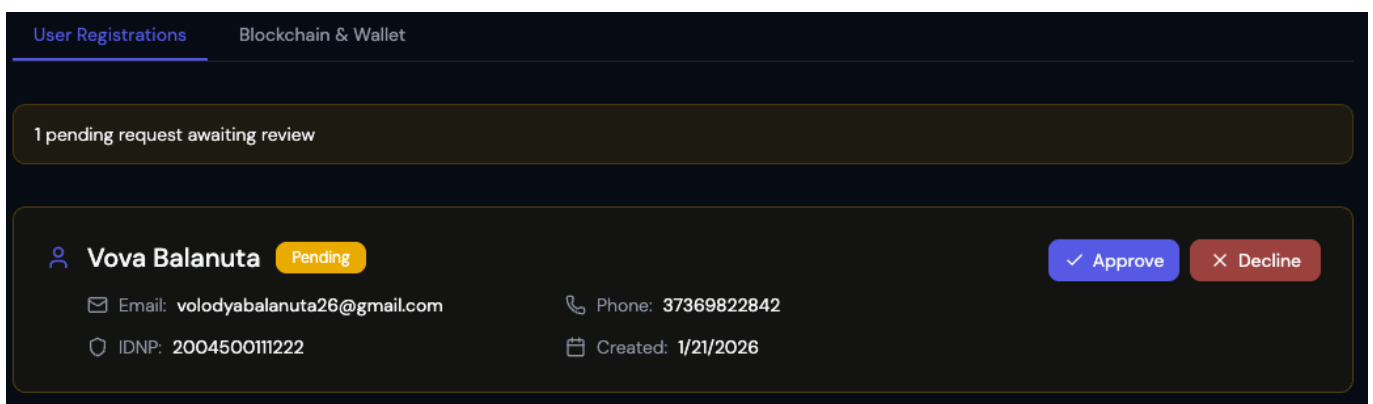


Figure 5.8 - Admin Console

The requests are listed in the table with the user information: Full Name, Email, Phone, IDNP and Action: Approve or Decline. If there are no requests, the page will display message: "No active requests" (Figure 5.9).

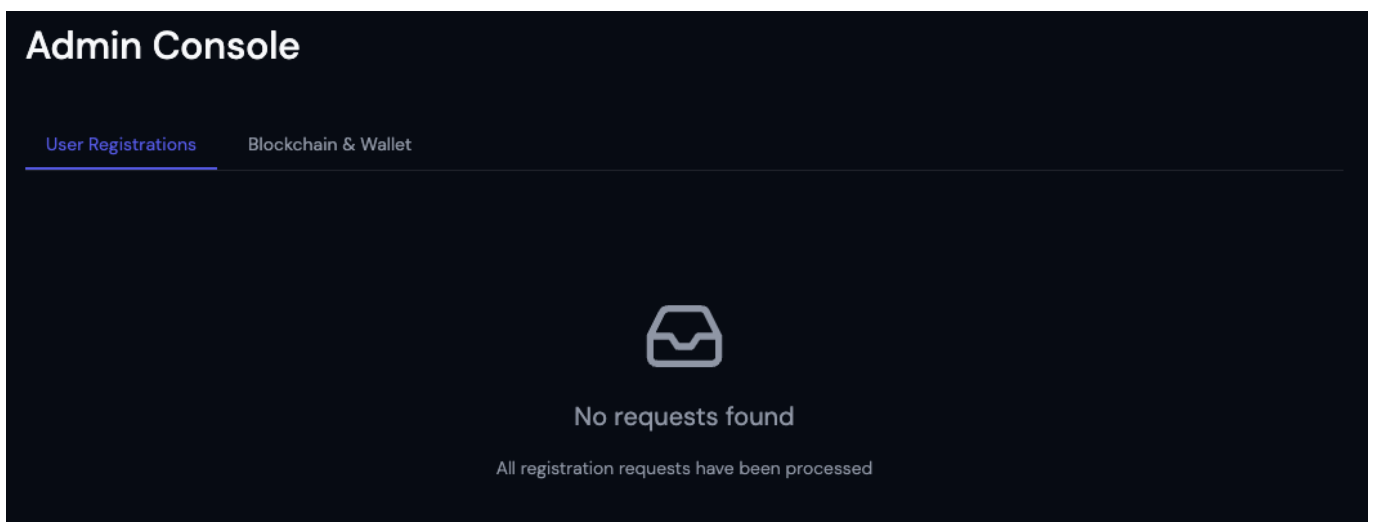


Figure 5.9 - Admin Console Empty

Administrator also have the ability to view the blockchain details such as: wallet information, which includes address, network, balance, total number of anchored documents (Figure 5.10).

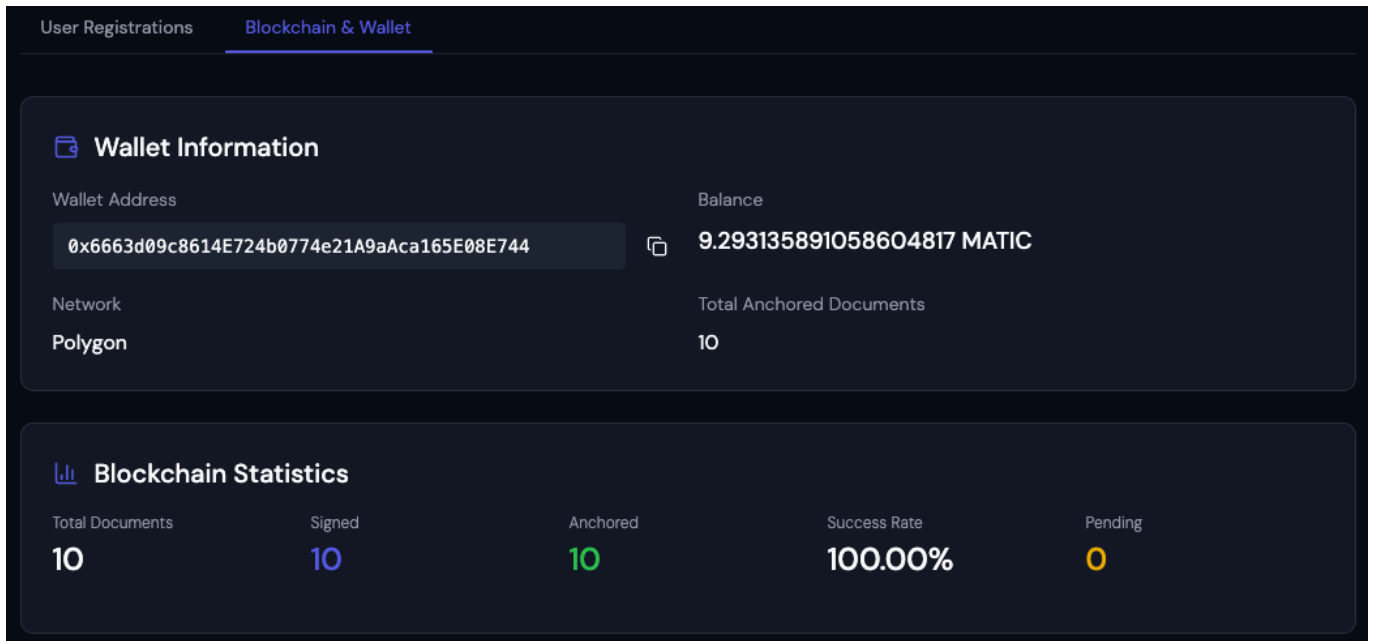


Figure 5.10 - Blockchain & Wallet

There is as well the list of all anchored documents and overall statistics (Figure 5.11).

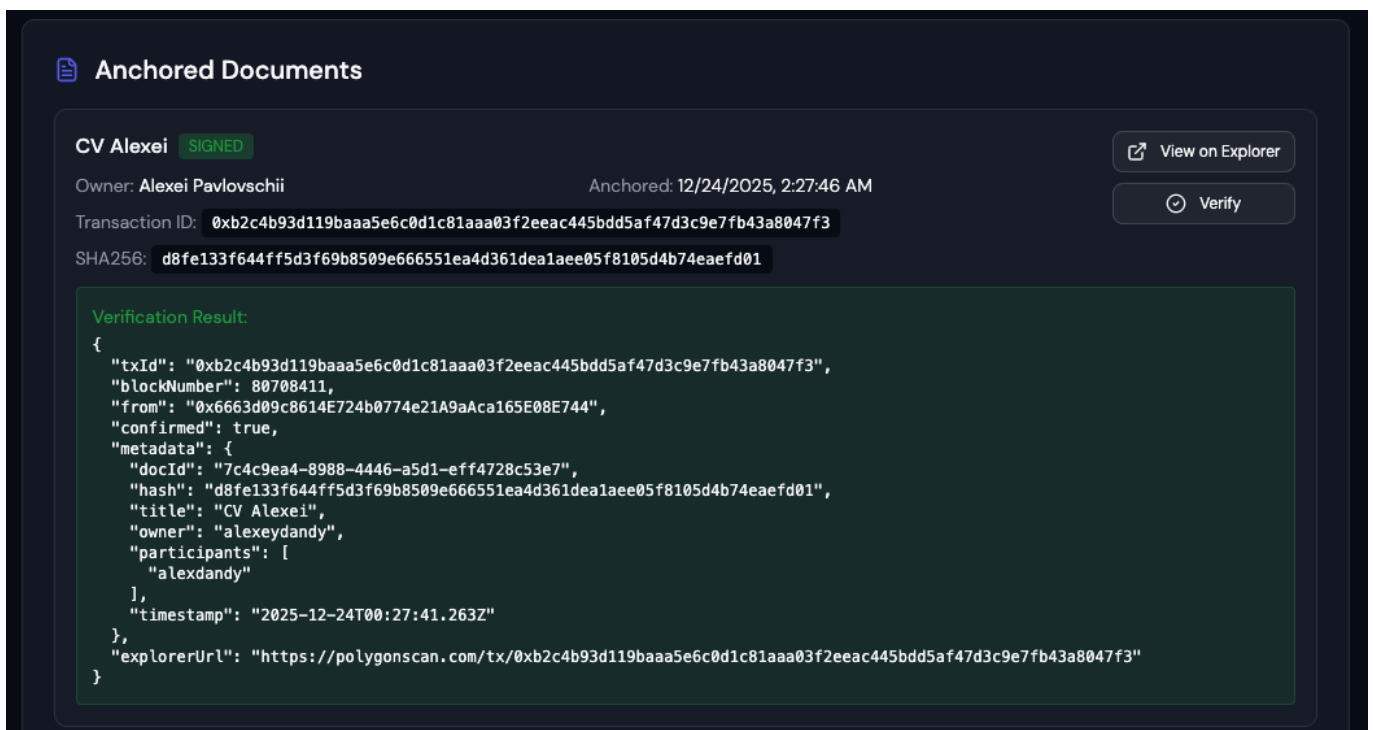


Figure 5.11 - Blockchain Documents Details

5.2.4 Additional registration steps

After that an email with the finish of the registration process is sent to the user (Figure 5.12).

BlockSign

Finish your BlockSign registration

Great news! Your registration request has been approved. Click the button below to finalize your account setup and complete your registration:

Open in BlockSign App

— or —

Continue on Web

If the buttons don't work, copy and paste this link into your browser:

https://blocksign.md/register/finish?token=gPHD-vGKKdVclhjb2YQoH4dvMpY3y_vsiEOptLNPN_9o&email=volodyabalanuta26%40gmail.com

This link expires in **30 minutes**. Please complete your registration before it expires.

This email was sent by BlockSign

If you have any questions, please contact our support team.

Figure 5.12 - Finish Registration Mail

The email contains a button which redirects user directly to finish registration process. There is as well the link, which can be manually used to paste in the browser URL field. Clicking the button or link will lead to the page of the last registration stage - Generating the mnemonic seed phrase and instructions to save it by the user to the safe place (Figure 5.13).

Attention

Never share the mnemonic phrase. Anyone with these words has full access to your account

- | | |
|-----------|------------|
| 1 shrimp | 2 club |
| 3 camp | 4 deer |
| 5 frozen | 6 denial |
| 7 enough | 8 meat |
| 9 attract | 10 young |
| 11 wish | 12 vicious |

 Copy to clipboard

I've saved my phrase

Store this mnemonic phrase in a safe place. You'll need it to access your account.

Figure 5.13 - Mnemonic Phrase

The phrase is generated on each refresh, but is valid only after clicking the button continue otherwise

it is not valid anywhere. Then the user can choose to set an additional PIN-code or remain only with mnemonic phrase (Figure 5.14).

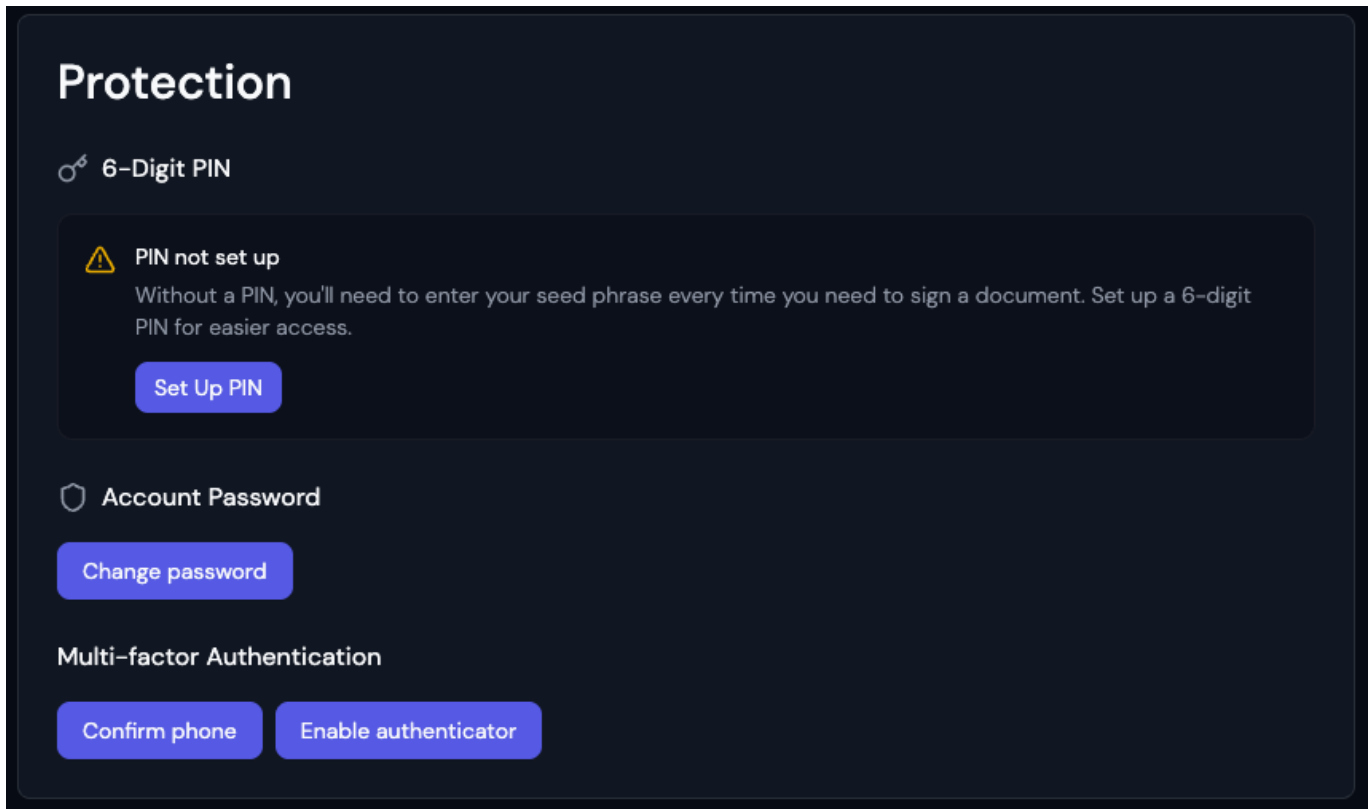


Figure 5.14 - Security Method Selection

5.2.5 Document Management Interface

The document management system provides users with comprehensive tools for uploading, signing, and managing digital documents. The interface is designed to streamline the document workflow while maintaining the highest security standards.

Dashboard and Navigation

Once authenticated, users are presented with a clean and intuitive dashboard that provides access to all document-related functionality. The dashboard displays recent document activity, pending signature requests, and quick access to document creation tools. Navigation system is designed to be intuitive, with clear visual indicators for different user roles and permissions. Authorized users can easily access their document library, create new documents, and manage their account settings.

Document Creation and Upload

The document creation process guides users through uploading PDF files and configuring signature requirements. The interface includes drag-and-drop functionality for easy file upload, along with validation to ensure only supported file formats are accepted.

Users can specify multiple participants for document signing, with each participant receiving noti-

fication emails containing the document and instructions for the signing process. The system automatically generates cryptographic hashes for document integrity verification.

Document Signing Workflow

The signing interface provides a secure environment for users to review and sign documents. Before signing, users can verify the document's integrity by comparing cryptographic hashes. The interface clearly displays document metadata, participant information, and signing status.

The signing process requires users to authenticate using their mnemonic phrase or PIN, ensuring that only authorized individuals can create valid signatures. Real-time feedback is provided throughout the signing process, including confirmation of successful signature creation and verification.

Document History and Verification

Users can access a comprehensive history of all documents they have created or signed. The interface provides detailed information about each document, including creation date, participants, signature status, and verification results.

The verification interface allows users to upload documents and verify their authenticity by checking cryptographic signatures and document integrity. This feature is available to both registered users and anonymous visitors, promoting transparency and trust in the digital signature process.

5.2.6 Responsive Design and User Experience

The BlockSign frontend implements a fully responsive design that adapts seamlessly to different screen sizes and devices. The application provides an optimal user experience across desktop computers, tablets, and mobile devices.

Mobile-First Approach

The design follows a mobile-first approach, ensuring that the core functionality is accessible and usable on smaller screens. Tailwind CSS breakpoints are used strategically to provide device-specific layouts and component arrangements.

Key responsive features include:

- Adaptive navigation menus that collapse on mobile devices
- Touch-friendly button sizes and interactive elements
- Optimized form layouts for mobile input
- Responsive image scaling and document previews
- Accessible typography and color contrast ratios

Cross-Browser Compatibility

The application is tested and optimized for modern web browsers, including Chrome, Firefox, Safari, and Edge. The codebase uses progressive enhancement techniques to ensure functionality across different browser versions while providing enhanced features for newer browsers.

5.2.7 Client-Side Security Implementation

The frontend architecture prioritizes security at every level, implementing multiple layers of protection for user data and cryptographic operations. All sensitive operations are performed client-side to minimize the exposure of private information.

Cryptographic Key Management

Private keys and mnemonic phrases are never transmitted to the server. All cryptographic operations, including key generation, signing, and verification, are performed within the user's browser using JavaScript cryptographic libraries.

The system implements secure storage mechanisms for sensitive data:

- Mnemonic phrases are stored temporarily in memory during active sessions
- PIN codes are encrypted using public keys before local storage
- Session tokens are managed securely with automatic expiration
- Private keys are derived from mnemonic phrases on-demand

Secure Communication

All communication between the frontend and backend uses HTTPS encryption. The application implements Content Security Policy (CSP) headers to prevent cross-site scripting attacks and other security vulnerabilities.

API requests are authenticated using JWT tokens with short expiration times, and refresh tokens are stored in secure HTTP-only cookies to prevent JavaScript-based attacks.

5.2.8 Internationalization and Localization

The application supports multiple languages to serve a diverse user base. The internationalization system uses i18next for dynamic language switching and locale-specific formatting. Currently supported languages include English (default), Romanian and Russian. The language selection interface allows users to switch between supported languages dynamically, with all text content, error messages, and user interface elements updating accordingly. The localization system handles date and time formatting according to locale preferences, number formatting, currency display, text direction support for future RTL language integration as well as culturally appropriate messaging/terminology.

5.2.9 Feature Accessibility by User Role

The BlockSign platform implements a role-based access control system that provides different levels of functionality based on user authentication status and administrative privileges.

Public Features (Unauthorized Users)

Unauthorized users have access to essential platform information and basic functionality without requiring registration:

- **Platform Information:** Access to landing page content, feature descriptions, and educational materials about digital signatures
- **User Registration:** Complete registration process including email verification and account request submission
- **Document Verification:** Upload and verify the authenticity of signed documents using cryptographic verification
- **Multi-language Support:** Switch between supported languages for better accessibility

Authenticated User Features

Registered and verified users gain access to comprehensive document management capabilities:

- **Document Creation:** Upload PDF documents and initiate signature workflows with multiple participants
- **Digital Signing:** Sign documents using cryptographic keys with mnemonic phrase or PIN authentication
- **Document History:** View complete history of created and signed documents with detailed metadata
- **Signature Management:** Track signature status for multi-party documents and receive notifications
- **Account Management:** Update profile information, manage security settings, and configure notification preferences
- **Participant Coordination:** Invite other users to sign documents and manage participant workflows
- **Document Verification:** Verify signatures on received documents and check document integrity
- **Secure Storage:** Access encrypted document metadata and signature records

5.2.10 Technology Integration and Performance

The frontend implementation leverages modern web technologies to provide optimal performance and user experience across different devices and network conditions. Key performance features include:

- **Code Splitting:** Dynamic imports and lazy loading for reduced initial bundle size
- **Image Optimization:** Automatic image compression and format selection for faster loading
- **Caching Strategy:** Strategic use of browser caching for static assets and API responses
- **Progressive Loading:** Incremental content loading to improve perceived performance

5.3 iOS Mobile Application

The BlockSign iOS application extends the platform's capabilities to mobile devices, providing users with a native, secure, and user-friendly experience for managing digital signatures on the go. Built using SwiftUI and following Apple's Human Interface Guidelines, the application maintains feature parity with the web platform while leveraging iOS-specific capabilities for enhanced security and usability.

5.3.1 Mobile Architecture

The iOS application follows the MVVM (Model-View-ViewModel) architectural pattern, ensuring clear separation of concerns and maintainable code structure. The architecture is designed to handle offline scenarios gracefully while maintaining cryptographic security standards.

Core Components

The application is structured around several key architectural components that work together to provide a seamless user experience. The Models layer defines data structures representing users, documents, signatures, and registration workflows, ensuring type-safe data handling throughout the application. ViewModels serve as the business logic layer, managing authentication flows, document operations, and registration processes while maintaining clean separation from the presentation layer. The Views layer consists of SwiftUI-based user interface components that provide intuitive navigation and interaction patterns optimized for mobile devices. Services provide utility modules for critical functions including API communication, biometric authentication, and cryptographic operations. Finally, the Utils layer includes helper functions for BIP-39 mnemonic generation, application theming, and configuration management, supporting the core functionality with reusable utilities.

5.3.2 Authentication and Security

The mobile application implements passwordless authentication using Ed25519 digital signatures, maintaining consistency with the web platform's security model. Users authenticate by signing cryptographic challenges with their private keys, which are securely derived from BIP-39 mnemonic phrases.

Biometric Integration

The iOS application leverages native biometric authentication capabilities to provide seamless and secure access. Face ID serves as the primary biometric authentication method on supported devices, offering quick and secure access without requiring manual credential entry. For devices without Face ID (Figure 5.15), Touch ID provides an alternative biometric option, ensuring all users can benefit from biometric security. Mnemonic phrases and private keys are stored securely using iOS Keychain Services, which provides hardware-backed encryption on devices with Secure Enclave. The application integrates deeply with the LocalAuthentication framework to handle biometric verification, including fallback scenarios and user cancellation gracefully.

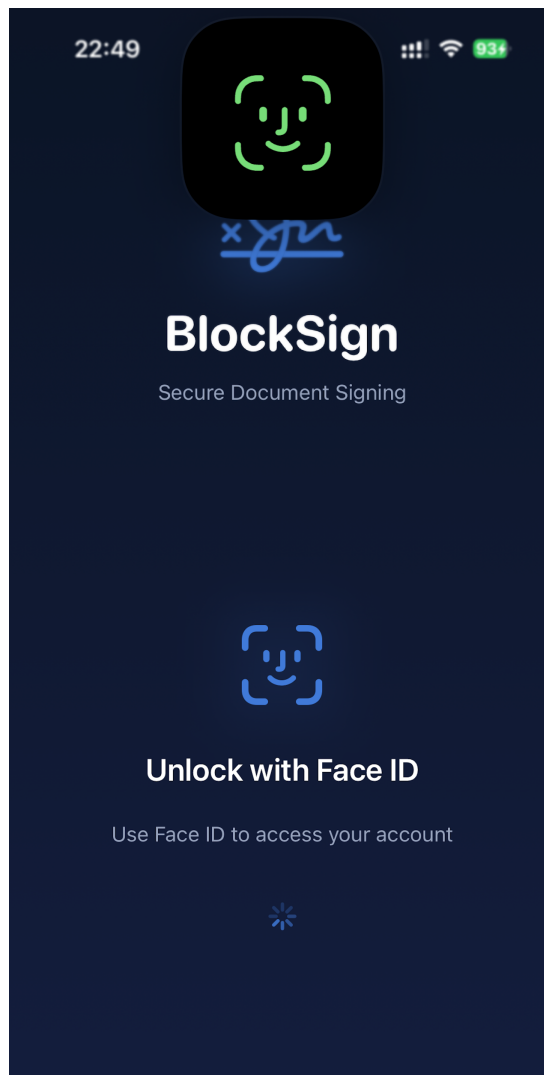


Figure 5.15 - iOS login screen with email entry

Cryptographic Implementation

The application implements client-side cryptography using native Swift libraries and carefully selected third-party packages to ensure maximum security and compatibility. The BIP-39 implementation handles mnemonic phrase generation and validation following Bitcoin Improvement Proposal 39 standards, ensuring compatibility with industry-standard wallet recovery mechanisms. For key derivation, the application uses SLIP-0010 to generate hierarchical deterministic Ed25519 keys from the mnemonic phrase, allowing for secure and reproducible key generation. Ed25519 signature generation and verification are implemented using a pure Swift implementation, eliminating external dependencies for this critical security function. Document integrity verification relies on SHA-256 hashing implemented through the Common-Crypto framework, providing fast and reliable hash computation. When users opt for additional security with PIN codes, the application employs PBKDF2 for key derivation, adding an extra layer of protection. All cryptographic operations are performed locally on the device, ensuring that private keys never leave the user's control.

5.3.3 Document Management

The mobile application provides comprehensive document management capabilities optimized for touch interfaces (Figure 5.16):

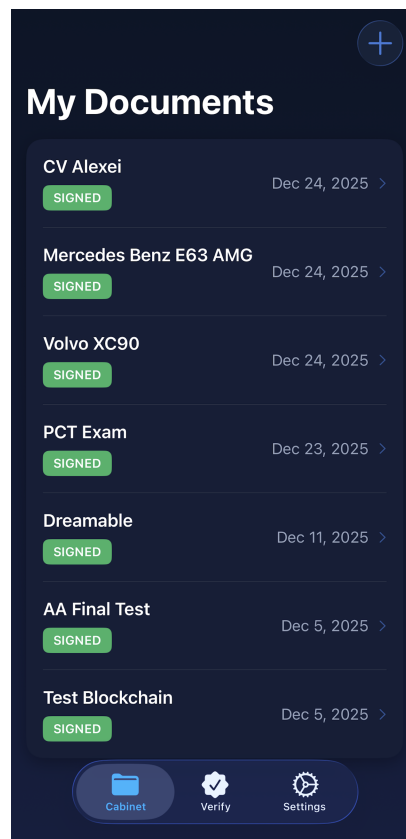


Figure 5.16 - My Documents dashboard with status indicators

Registration Flow

The application provides a streamlined registration experience with deep link support for email verification:

Multi-Step Registration Process

The registration workflow guides users through several verification steps:

1. **Email Entry:** Initial email address submission with validation
2. **OTP Verification:** Six-digit code entry with automatic submission upon completion
3. **User Details:** Personal information including full name, phone number, and IDNP
4. **Request Submission:** Pending approval status with clear user feedback
5. **Email Verification:** Deep link activation via email notification
6. **Mnemonic Generation:** Client-side seed phrase creation with secure storage instructions
7. **Account Completion:** Final setup including public key registration and signature verification

Deep Link Integration

The application implements custom URL scheme handling for seamless email-to-app transitions. A custom `blocksign://` protocol enables deep linking, allowing email notifications to open the app directly to specific screens. The application securely extracts registration tokens from deep link URLs, validating them before processing to prevent malicious link exploitation. State management ensures proper handling of the app state during deep link activation, whether the app is in the foreground, background, or not running. For users who haven't installed the app, the system provides a web-based completion option as a fallback, ensuring the registration process can be completed regardless of platform availability.

Document Creation

Users can create new signing requests directly from their mobile devices (Figure 5.17).

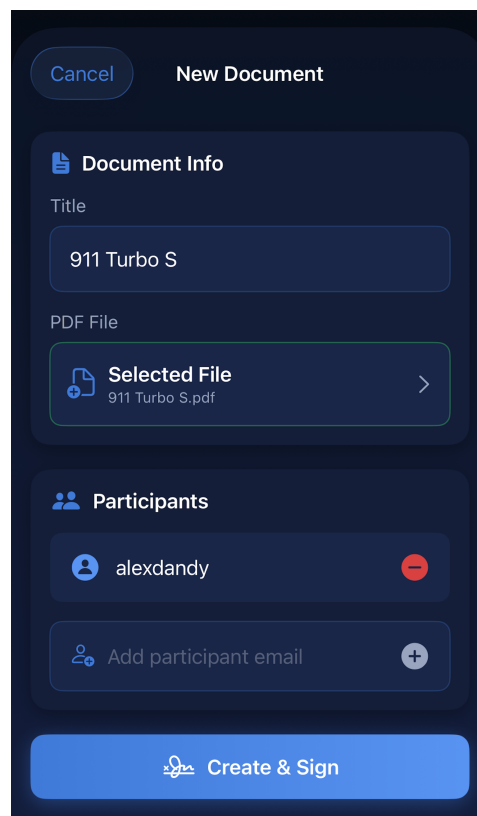


Figure 5.17 - Create new document interface with participant management

The native file picker integration allows for seamless PDF selection from the device's file system, iCloud Drive, or other configured storage providers. Participant management functionality enables adding multiple signers by entering their email addresses or usernames, with validation to ensure all required information is correctly formatted. The application automatically calculates the SHA-256 hash of selected documents, providing visual feedback during the computation process to keep users informed. Client-side payload signing uses the securely stored private key to generate the initial signature, maintaining the zero-trust security model. Real-time progress indicators display upload status during file transfer to the server,

giving users confidence that the operation is proceeding correctly.

Document Dashboard

The main dashboard presents users with an organized and intuitive view of their documents. Documents are displayed in a chronologically ordered list, making it easy to find recent items and track document history over time. Visual status badges provide immediate feedback on document state, clearly distinguishing between pending, signed, and rejected documents through color-coded indicators. The interface supports pull-to-refresh gestures, allowing users to update their document list with an intuitive downward swipe motion. Swipe gestures enable quick actions on individual documents, streamlining common operations without requiring multiple navigation steps. The dashboard includes fast search functionality that filters documents by title or status, helping users quickly locate specific items even in large document collections.

Document Details and Signing

The document details screen provides comprehensive information about each document (Figure 5.18):

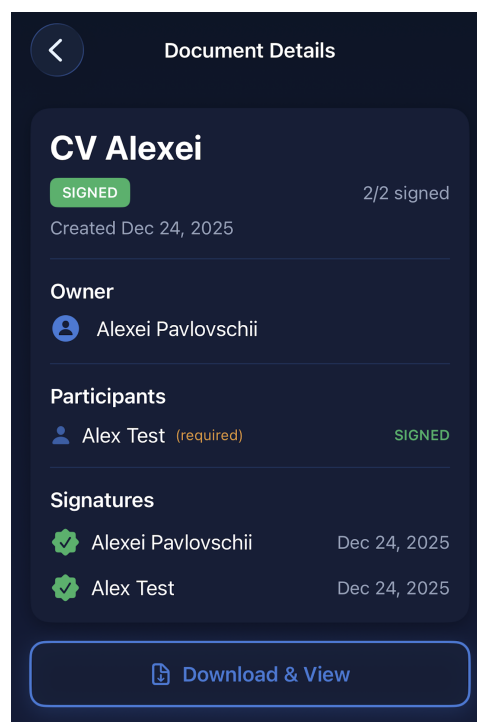


Figure 5.18 - Document details with owner, participants, and signatures

The document details screen provides comprehensive information about each document, serving as the central hub for reviewing and signing operations. PDF rendering enables full document review before signing, allowing users to verify content accuracy directly within the application. The interface prominently displays and verifies the document integrity hash, ensuring users can confirm they're viewing the exact document that will be signed. Clear visibility of all required signers and their current signing status helps users track workflow progress and identify any pending actions. Face ID or Touch ID biometric

confirmation is required before signature authorization, adding a critical security checkpoint that prevents unauthorized signing. Local cryptographic signing provides immediate feedback upon completion, with the signed payload securely transmitted to the server. Real-time synchronization of signature status across participants ensures all users have up-to-date information about the document's current state in the signing workflow.

5.3.4 Document Verification

The application includes a public verification feature accessible without authentication (Figure 5.19):

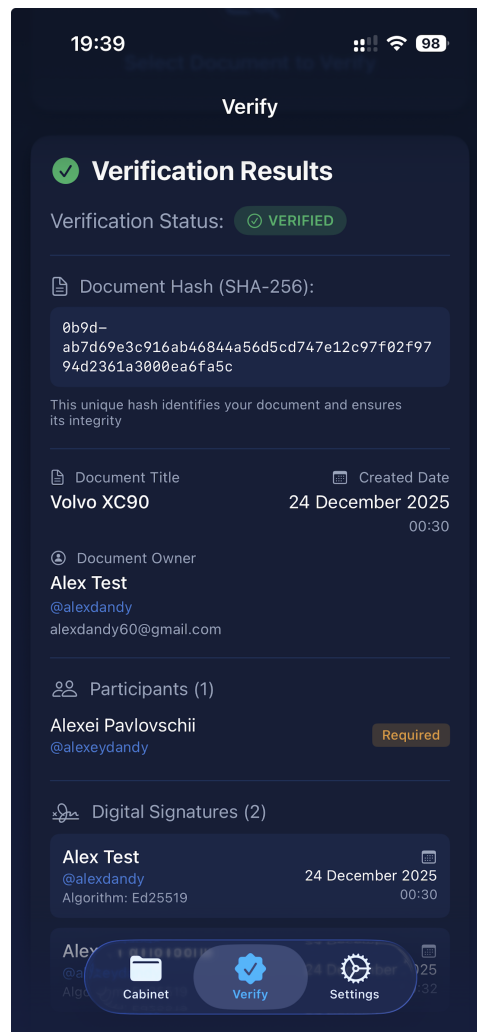


Figure 5.19 - Document verification interface

The application includes a public verification feature that requires no authentication, making document verification accessible to anyone who needs to validate signatures. Users begin by selecting PDF documents for verification through the native file picker, with support for documents from various sources including email attachments and cloud storage. Automatic SHA-256 computation runs immediately upon file selection, calculating the document hash for comparison with stored signature payloads. Cryptographic validation of all signatures confirms the authenticity of each signer, verifying that signatures were created by

the claimed private keys and haven't been tampered with. Detailed display of all signers and their individual verification status provides transparency about who signed the document and when. Optional blockchain anchor verification offers an additional layer of trust by confirming that the document hash was recorded on the blockchain at a specific timestamp. The comprehensive breakdown of verification status presents all validation results in a clear, user-friendly format that non-technical users can understand.

5.3.5 User Interface Design

The iOS application follows Apple's Human Interface Guidelines while maintaining brand consistency with the web platform. The design system incorporates both dark and light mode support with dynamic color adaptation that responds to system-wide appearance settings. Typography utilizes the San Francisco font family with accessibility-focused sizing that scales appropriately based on user preferences. Consistent padding and margins follow an 8-point grid system throughout the application, creating visual harmony and predictable spacing patterns. SF Symbols integration provides native iOS iconography that matches the system look and feel while supporting dynamic sizing and theming. Smooth transitions powered by SwiftUI's built-in animation system create a polished user experience with fluid state changes and view transitions.

The application adapts seamlessly to different iOS device form factors through responsive layout techniques. iPhone optimization provides compact layouts specifically designed for smaller screens, maximizing screen real estate while maintaining usability. iPad support features expanded layouts with centered content for better readability on larger displays, preventing awkward edge-to-edge stretching. Orientation handling ensures adaptive designs work equally well in both portrait and landscape modes, automatically adjusting layout constraints as needed. Proper handling of notches and home indicators respects safe areas on all device models, preventing interface elements from being obscured by hardware features. Dynamic Type support allows the interface to scale with user-selected text sizes, ensuring accessibility for users with visual impairments who require larger fonts.

5.3.6 Settings and Account Management

The settings interface provides access to account configuration and security options (Figure 5.20). The settings interface provides comprehensive access to account configuration and security options. Users can view and edit their profile information including email addresses, full names, and usernames, with in-line validation ensuring data integrity. Security settings grant access to recovery phrase viewing, protected by biometric authentication to prevent unauthorized access to this sensitive information. Biometric configuration options allow users to enable or disable Face ID or Touch ID authentication based on their security preferences and device capabilities. The interface displays the current app version and build information, useful for troubleshooting and ensuring users are running the latest release. A secure logout function performs local data cleanup, removing cached credentials and session information while maintaining necessary

persistent data.

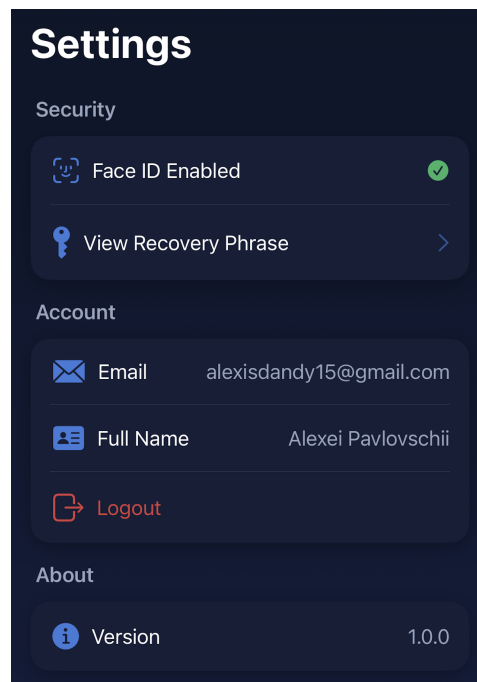


Figure 5.20 - Settings screen with security and account management

5.3.7 Offline Capabilities and Caching

The mobile application implements intelligent caching strategies to support offline usage scenarios. Document metadata caching provides local storage of document information, enabling offline viewing of document lists and basic details even without network connectivity. User profile caching maintains persistent storage of user details, reducing unnecessary API calls and improving performance when displaying account information. Strategic caching of non-sensitive API responses reduces server load and improves response times for frequently accessed data. An offline queue system captures pending operations and automatically executes them when network connectivity returns, ensuring no user actions are lost during temporary disconnections. Clear visual indicators provide feedback about online and offline status, helping users understand when features that require connectivity may be unavailable.

5.3.8 Error Handling and User Feedback

The application provides comprehensive error handling and user feedback mechanisms throughout the user journey. Network errors trigger clear messaging when connectivity issues occur, explaining the problem and suggesting resolution steps such as checking WiFi or cellular data. Validation errors appear inline with form inputs, providing immediate feedback about data format requirements without requiring form submission. Cryptographic failures receive detailed error messages that explain signature operation issues while maintaining user-friendly language for non-technical audiences. API errors are translated from technical server responses into user-friendly messages that explain what went wrong and suggest

corrective actions. Progress indicators display for all long-running operations, preventing user confusion during processes like file uploads or signature verification. Success confirmations combine visual feedback with haptic responses, providing multi-sensory confirmation when important actions complete successfully.

5.3.9 App Store Compliance

The application meets all Apple App Store requirements and guidelines to ensure successful approval and distribution. Encryption export compliance is properly declared through Info.plist entries that document the cryptographic algorithms used, satisfying US export regulations for encryption technology. A clear privacy policy discloses all data collection and usage practices, explaining what information the app accesses and how it's used to provide functionality. App Transport Security enforces HTTPS for all network communication, preventing insecure data transmission and protecting user information in transit. Background modes are used appropriately and only when necessary, with proper declarations for any background processing capabilities. Comprehensive TestFlight beta testing validates functionality across multiple device types and iOS versions before production release, identifying and resolving issues before they impact general users.

5.4 DevOps & Deployment

The BlockSign platform implements a robust DevOps pipeline that ensures reliable deployments, continuous integration, and production environment stability. The infrastructure is designed for scalability, security, and maintainability, leveraging Amazon Web Services (AWS) for cloud infrastructure and Xcode Cloud for mobile application deployment.

5.4.1 Cloud Infrastructure

The production environment is hosted on AWS, utilizing multiple services to create a secure, scalable, and reliable infrastructure. Amazon EC2 (Elastic Compute Cloud) instances provide the computing resources for hosting the back-end application, offering flexible instance types that can be scaled vertically or horizontally based on demand. An Elastic IP address ensures the production server maintains a consistent public IP address even when instances are replaced or restarted, simplifying DNS configuration and maintaining stable endpoint addresses for client applications.

Document storage is handled through Amazon S3 (Simple Storage Service), which provides highly durable and scalable object storage for uploaded PDF files. S3's built-in redundancy and versioning capabilities ensure document availability and enable recovery from accidental deletions. Email delivery relies on Amazon SES (Simple Email Service) for sending OTP codes, registration notifications, and document signing invitations, providing reliable email delivery with high deliverability rates and detailed sending metrics.

AWS IAM (Identity and Access Management) controls access to AWS resources, implementing the principle of least privilege by granting services and users only the permissions they require. IAM roles

enable EC2 instances to securely access S3 buckets and SES without storing long-term credentials, while IAM policies define granular permissions for different components of the infrastructure.

5.4.2 Mobile Application Deployment

The iOS application utilizes Xcode Cloud for automated building and deployment, integrating seamlessly with the GitHub repository. Xcode Cloud automatically builds the application whenever changes are pushed to the main branch, running unit tests and generating signed builds for TestFlight distribution. This automated workflow eliminates manual build processes and ensures consistent build environments, reducing the potential for environment-specific build issues. TestFlight enables beta testing with both internal and external testers before production release, providing valuable feedback and identifying issues before they reach the App Store.

5.4.3 Containerization Strategy

The back-end application is fully containerized using Docker, enabling consistent deployment across development, staging, and production environments.

Docker Configuration

The Docker setup includes optimized multi-stage builds that significantly reduce image sizes while improving security posture. The base image uses Node.js on Alpine Linux, providing a minimal footprint that includes only essential system libraries and reducing the attack surface. The build stage handles compilation of TypeScript source code to JavaScript, installing only production dependencies and excluding development tools from the final image. The production stage creates a minimal runtime image containing only necessary files for application execution, further reducing image size and potential vulnerabilities. Strategic layer ordering enables efficient rebuilds by placing frequently changing code layers after stable dependency layers, improving build cache utilization. Automated vulnerability scanning of container images runs during the build process, identifying and flagging security issues in both base images and application dependencies before deployment.

Docker Compose Orchestration

The application uses Docker Compose for orchestrating multiple services in both local development and production deployment scenarios. Service definitions specify the Node.js application container along with the PostgreSQL database and an optional Redis cache for session management and API response caching. Network configuration creates isolated internal networks that enable secure service-to-service communication while restricting external access to only necessary entry points. Volume management provides persistent storage for database data, while uploaded documents are stored in Amazon S3 rather than local volumes, providing scalable and durable storage with built-in redundancy. Environment variables are securely injected through environment files that remain outside version control, protecting sensitive configuration data including AWS credentials while allowing easy environment-specific customization. Health

checks monitor container status and automatically restart failed services, improving system reliability and reducing manual intervention requirements.

5.4.4 Continuous Integration and Deployment

The project implements automated CI/CD pipelines using GitHub Actions:

CI/CD Workflow

1. **Code Commit:** Developer pushes changes to GitHub repository
2. **Automated Testing:** Unit tests, integration tests, and linting checks
3. **Build Process:** Compilation and bundling of application code
4. **Container Build:** Docker image creation with proper tagging
5. **Image Push:** Upload to container registry
6. **Deployment Trigger:** Automatic deployment to production server
7. **Health Verification:** Post-deployment health checks and rollback on failure

GitHub Actions Configuration

The deployment workflow leverages GitHub Actions for automated continuous deployment with comprehensive safety checks. Trigger conditions enable automated deployment whenever changes are pushed to the main branch, ensuring the production environment stays synchronized with the latest stable code. Environment secrets provide secure storage of deployment credentials and API keys, including AWS access keys and SSH credentials, preventing sensitive information from being exposed in repository code or logs. SSH deployment connects to the EC2 instance for executing deployment commands and transferring updated container images. Zero-downtime deployment implements rolling updates with health checks that verify new container instances before removing old ones, ensuring continuous service availability. Rollback capability provides automatic reversion to the previous version if deployment health checks fail, minimizing the impact of problematic releases and maintaining system stability.

5.4.5 Database Management

Database operations are managed through Prisma ORM with comprehensive migration support:

Migration Strategy

Database operations follow a comprehensive migration strategy that ensures data integrity throughout the application lifecycle. Schema versioning through Git-tracked migration files creates reproducible database states that can be reliably deployed across multiple environments. Automated migrations execute as part of the deployment pipeline, applying pending schema changes before starting updated application code. Pre-migration database backups provide safety nets, allowing quick recovery if migration issues cause data corruption or unexpected schema changes. Rollback support enables reverting problematic migrations, though this capability is used cautiously due to the complexity of reversing data transformations. Seeding functionality populates initial data for development and testing environments, ensuring consistent baseline

data for quality assurance and local development work.

Database Security

Database security implements multiple layers of protection to safeguard sensitive user and document data. Connection security mandates TLS/SSL encryption for all database connections, preventing man-in-the-middle attacks and ensuring data confidentiality during transmission. Access control uses role-based database user permissions, limiting each application component to only the database operations it requires and reducing the potential impact of security breaches. Credential management relies on environment variable-based injection rather than hardcoded values, keeping database passwords out of source code and configuration files committed to version control. Automated backup scheduling runs regular database backups with configurable retention policies, balancing storage costs against data recovery requirements. Performance monitoring tracks query execution times and identifies slow queries, enabling optimization of database access patterns and preventing performance degradation as data volumes grow.

5.4.6 Environment Configuration

The application supports multiple deployment environments with distinct configurations tailored to each use case. Development environments provide local development capabilities with hot-reloading that automatically refreshes the application when code changes, along with verbose debug logging that aids troubleshooting. Staging environments mirror production configuration as closely as possible, providing a pre-production testing ground where new features can be validated under production-like conditions before general release. Production environments use optimized settings including minified assets, reduced logging verbosity, and enhanced security measures to maximize performance and protect sensitive data. Security considerations mandate that sensitive values remain in environment files that are explicitly excluded from version control, preventing accidental exposure of credentials or API keys in commit history.

5.4.7 Monitoring and Logging

Production deployments include comprehensive monitoring and logging infrastructure that provides visibility into application health and performance. Application monitoring utilizes dedicated health endpoints that respond to HTTP requests, enabling both automated monitoring systems and manual health checks to verify service availability. Performance metrics track response times for API endpoints, identifying slow operations and enabling proactive optimization before users experience degraded performance. Error tracking automatically captures and reports application exceptions, providing detailed stack traces and context information that accelerate debugging and resolution. Resource monitoring tracks CPU utilization, memory consumption, and disk usage patterns, alerting operations teams before resource exhaustion causes service disruption. External uptime monitoring services poll the application from multiple geographic locations, detecting regional outages and verifying global availability.

The logging strategy implements structured logging throughout the application, with JSON-formatted

log entries that enable efficient machine parsing and analysis. Appropriate log levels distinguish between debug information useful during development, informational messages about normal operations, warnings about potential issues, and errors requiring immediate attention. Centralized log aggregation collects logs from all application instances into a unified system, simplifying troubleshooting across distributed deployments and enabling correlation of events across multiple services. Time-based log rotation and archival balance the need for historical data against storage costs, retaining recent logs for quick access while archiving older logs to cheaper storage tiers. Security logging creates comprehensive audit trails for authentication attempts, authorization decisions, and sensitive operations, supporting both security investigations and compliance requirements.

5.4.8 Security Hardening

Production deployments implement multiple layers of security hardening to protect against common attack vectors. HTTPS enforcement through SSL/TLS certificates with automatic renewal ensures all client-server communication remains encrypted, with modern cipher suites that balance security and performance. Firewall configuration restricts port access to only necessary services while implementing IP whitelisting for administrative interfaces, reducing the attack surface available to potential attackers. Rate limiting applies request throttling to prevent abuse, protecting against denial-of-service attacks and automated credential stuffing attempts while allowing legitimate high-volume users to operate normally. Security headers including Content Security Policy, HTTP Strict Transport Security, and X-Frame-Options protect against cross-site scripting, protocol downgrade attacks, and clickjacking vulnerabilities. Automated dependency scanning continuously monitors npm packages for known vulnerabilities, alerting when security updates become available and enabling proactive patching before exploits emerge. Container security measures include running processes as non-root users and applying minimal file system permissions, limiting the potential damage from container escape vulnerabilities.

5.4.9 Scalability Considerations

The infrastructure is designed from the ground up to support horizontal scaling as user demand grows over time. The stateless application design ensures servers maintain no local state, storing all session data in shared storage systems like Redis, which enables adding or removing application instances without affecting user sessions or requiring complex synchronization. Load balancing distributes incoming traffic across multiple application instances, improving both performance and reliability by preventing any single server from becoming a bottleneck. Database scaling support includes provisions for read replicas that handle query operations while the primary database handles writes, along with connection pooling that efficiently manages database connections across multiple application instances. Caching strategy implementation through Redis integration stores frequently accessed data in memory, dramatically reducing database load and improving response times for common queries. CDN integration distributes static assets

like images, stylesheets, and JavaScript files through geographically distributed content delivery networks, reducing latency for users worldwide. The modular architecture remains microservices-ready, with clear boundaries between components that facilitate future decomposition into separate services if scaling requirements demand more granular deployment control.

5.4.10 Disaster Recovery

Comprehensive disaster recovery procedures ensure business continuity even in the face of catastrophic failures. The backup strategy implements automated daily backups of both database content and critical configuration files, with Amazon S3 providing durable storage for backup archives with configurable retention periods that balance recovery capabilities against storage costs. Regular backup verification through periodic restoration tests ensures backup integrity and validates that recovery procedures work as documented, preventing unpleasant surprises during actual disaster scenarios. S3's built-in geo-redundant storage replicates backups across multiple AWS availability zones, protecting against regional disasters like data center failures or natural disasters that could destroy primary infrastructure. Documented Recovery Time Objectives establish target timeframes for restoring service after various failure scenarios, guiding infrastructure investments and operational procedures. Recovery Point Objectives define maximum acceptable data loss thresholds, informing backup frequency and transaction log retention policies. A detailed incident response plan documents step-by-step procedures for handling different types of outages, ensuring operations teams can execute recovery efficiently even under stress, with clear escalation paths and communication protocols. The use of Elastic IP addresses simplifies recovery by allowing quick reassignment to replacement EC2 instances without requiring DNS propagation delays.

CONCLUSIONS

The work carried out in this project demonstrates that secure, scalable, and verifiable document management can be achieved through the careful integration of modern cryptographic techniques, structured workflows, and practical user interfaces. By replacing traditional password-based authentication with Ed25519 key pairs and challenge–response mechanisms, the system eliminates a major vulnerability of centralized platforms while improving usability and security.

The design of the registration process, supported by email-based verification and administrative approval, ensures that only legitimate users gain access to the platform. At the same time, the introduction of digital signatures over canonical payloads provides a strong guarantee of document integrity and participant accountability. Each document is uniquely identified by its cryptographic hash, which allows all stakeholders to independently verify its authenticity.

From a functional perspective, the project delivers a complete document signing platform with user registration, passwordless authentication, document creation with multi-participant workflows, and comprehensive signature collection. The system supports three distinct deployment environments: a web application built with Next.js and React, a native iOS mobile application developed with SwiftUI, and a robust back-end API implemented in Node.js with Express and PostgreSQL. Email notifications powered by Amazon SES keep participants informed throughout the signing process, while document storage on Amazon S3 ensures durability and scalability. The platform includes blockchain anchoring on Polygon network, providing immutable proof of document existence and timestamping for transparency and legal validity.

The implementation confirms the feasibility of using modern cryptographic libraries and structured database schemas to create secure document workflows with enterprise-grade infrastructure. The system operates in production on AWS infrastructure, with EC2 instances handling backend computing, Elastic IP ensuring stable connectivity, and comprehensive DevOps practices including automated CI/CD through GitHub Actions and mobile deployment via Xcode Cloud. The iOS application leverages native biometric authentication through Face ID and Touch ID, providing seamless security that matches user expectations for mobile applications.

In conclusion, this project achieves its objective of creating a production-ready system for passwordless user authentication and legally binding document signing. It demonstrates the successful combination of cryptographic security with intuitive user workflows across multiple platforms, establishing a complete solution that addresses real-world document management needs. The platform integrates modern cloud infrastructure, blockchain technology, and mobile-first design principles to deliver a secure, accessible, and verifiable document signing experience suitable for organizational deployment.

BIBLIOGRAPHY

- [1] INTERPOL, *Identity and travel document fraud*. [Online]. Available: <https://www.interpol.int/en/Crimes/Counterfeit-currency-and-security-documents/Identity-and-travel-document-fraud>
- [2] U.S. Customs and Border Protection, *Cincinnati CBP intercepts thousands of fraudulent and counterfeit documents during FY2023*. [Online]. Available: <https://www.cbp.gov/newsroom/local-media-release/cincinnati-cbp-intercepts-thousands-fraudulent-and-counterfeit>
- [3] International Council on Archives, *Emergency management and disaster preparedness: A manual for protecting archives (2024)*. [Online]. Available: https://www.ica.org/app/uploads/2024/10/Emergency-Management-and-Disaster-Preparedness-Manual_FINAL_VERSION_PDFA.pdf?utm_source=chatgpt.com
- [4] Geoengineer.org, *Cologne archive collapse: How a 0.6m void led to germany's costliest construction failure (2025)*. [Online]. Available: <https://www.geoengineer.org/news/cologne-archive-collapse-how-a-06m-void-led-to-germanys-costliest-construction-failure>
- [5] Verizon, *2024 data breach investigations report (DBIR)*. [Online]. Available: <https://www.verizon.com/business/resources/reports/2024-dbir-data-breach-investigations-report.pdf>
- [6] IBM, *Cost of a data breach 2025 (live report page)*. [Online]. Available: <https://www.ibm.com/reports/data-breach>
- [7] IBM, *Cost of a data breach 2024*. [Online]. Available: <https://www.ibm.com/think/insights/cost-of-a-data-breach-2024-financial-industry>
- [8] Reuters — FBI, *Complaints about ransomware attacks on u.s. infrastructure rose 9% in 2024*. [Online]. Available: <https://www.reuters.com/world/us/complaints-about-ransomware-attacks-us-infrastructure-rise-9-fbi-says-2025-04-23/>
- [9] WIRED, *DigiNotar files for bankruptcy in wake of devastating hack*. [Online]. Available: <https://www.wired.com/2011/09/diginotar-bankruptcy/>
- [10] HIPAA Journal, *Verizon 2024 DBIR: 70% of healthcare breaches involved internal actors (sector-specific summary)*. [Online]. Available: <https://www.hipaajournal.com/verizon-2024-data-breach-investigations-report/>
- [11] EUR-Lex, *GDPR (regulation (EU) 2016/679) official text*. [Online]. Available: <https://eur-lex.europa.eu/eli/reg/2016/679/oj/eng>

- [12] EUR-Lex, *eIDAS (regulation (EU) no 910/2014) consolidated*. [Online]. Available: <https://eur-lex.europa.eu/eli/reg/2014/910/2024-10-18/eng>
- [13] OWASP, *Top 10 (2021)*. [Online]. Available: <https://owasp.org/Top10/>
- [14] OWASP, *A01: Broken access control (2021)*. [Online]. Available: https://owasp.org/Top10/A01_2021-Broken_Access_Control/
- [15] World Economic Forum, *Global risks report 2023*. [Online]. Available: <https://www.weforum.org/publications/global-risks-report-2023>
- [16] UNESCO, *Credentials fraud now a global threat for universities*. [Online]. Available: <https://etico.iiep.unesco.org/en/credentials-fraud-now-global-threat-universities>
- [17] European Commission, *Digital transformation of SMEs*. [Online]. Available: https://commission.europa.eu/projects/digital-transformation-smes_en
- [18] European Commission, *eIDAS regulation*. [Online]. Available: <https://digital-strategy.ec.europa.eu/en/policies/eidas-regulation>
- [19] NCFA Canada, *The future of remote online notarization: Advancements in security and efficiency*. [Online]. Available: <https://ncfacanada.org/the-future-of-remote-online-notarization-advancements-in-security-and-efficiency>
- [20] ACCENTURE, *State of cybersecurity resilience 2023*. [Online]. Available: <https://www.accenture.com/us-en/insights/security/state-cybersecurity>
- [21] Kadir Canoz, *Use of electronic medical records in the digital healthcare system and its role in communication and medical information sharing among healthcare professionals*. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S2352914823002198>
- [22] Grand View Horizon, *Global digital signature market size & outlook, 2024-2030*. [Online]. Available: <https://www.grandviewresearch.com/horizon/outlook/digital-signature-market-size/global>
- [23] Acumen, *Digital signature market size growing at 35.4% CAGR*. [Online]. Available: <https://www.globenewswire.com/news-release/2023/01/30/2598004/0/en/Digital-Signature-Market-Size-Growing-at-35-4-CAGR-Set-to-Reach-USD-48-4-Billion-by-2030.html>
- [24] Electro IQ, *E-signature and digital document statistics*. [Online]. Available: <https://electroiq.com/stats/e-signature-and-digital-document-statistics>
- [25] DoxyChain, *The power of blockchain notarization*. [Online]. Available: <https://www.doxychain.com/blog/the-power-of-blockchain-notarization-securing-digital-assets-against-deepfakes-and-fraud>
- [26] Deeksha Uikey, *A blockchain-based digital notary system provides reliable and tamper-proof timestamping and verification services for digital documents: A review*. [Online]. Available: <https://>

- pdfs.semanticscholar.org/c1d0/ee0aa761ba214d5c4c899147aab358eace25.pdf] [https://powerpatent.com/blog/blockchain-for-document-verification-and-notarization
- [27] Tejas Chandrakant Mhapankar, *Future trends in blockchain-based notarization: From proof of existence to full document validation*. [Online]. Available: <https://harbinengineeringjournal.com/index.php/journal/article/download/4322/2423/6793>
- [28] Domenico Tortola, *Scalable data notarization leveraging hybrid DLTs*. [Online]. Available: <https://arxiv.org/abs/2501.04571>
- [29] SutiSoft, *Role of multi-factor authentication in eSignatures for advanced security*. [Online]. Available: <https://www.sutisoft.com/blog/esignature-multifactor-authentication-security/>
- [30] PingIdentity, *Eight benefits of multi-factor authentication (MFA)*. [Online]. Available: <https://www.pingidentity.com/en/resources/blog/post/eight-benefits-mfa.html>
- [31] Uplevel Systems, *Top 10 benefits of implementing multi-factor authentication (MFA) for small businesses*. [Online]. Available: <https://www.uplevelsystems.com/blog/top-10-benefits-of-implementing-mfa-for-small-businesses>
- [32] ProNotary, *How law firms are using remote online notarization to increase client satisfaction*. [Online]. Available: <https://pronotary.com/blog/how-law-firms-are-using-remote-online-notarization-to-increase-client-satisfaction>
- [33] Wikipedia, *ENotary*. [Online]. Available: <https://en.wikipedia.org/wiki/ENotary>
- [34] Wikipedia, *Notario (firm)*. [Online]. Available: [https://en.wikipedia.org/wiki/Notario_\(firm\)](https://en.wikipedia.org/wiki/Notario_(firm))
- [35] Wikipedia, *Privacy and blockchain*. [Online]. Available: https://en.wikipedia.org/wiki/Privacy_and_blockchain
- [36] Alexei Pavlovskii, *Blocksign server*. [Online]. Available: <https://github.com/AlexDandy77/BlockSign-Server>
- [37] Vladimir Vitkovskii, *Blocksign front-end*. [Online]. Available: <https://github.com/NeValetik/BlockSign-FrontEnd>
- [38] Alexei Pavlovskii, *Blocksign ios*. [Online]. Available: <https://github.com/AlexDandy77/BlockSign-iOS>